Modelling and Control of an Automated Anti-Aircraft Defense System (AADS) Turret

MKM506E - Modeling and Control of Mechatronic Systems
Prof. Onur Keskin

İTÜ Master's Candidate:
Kerem Açıkgöz - 518241007
Elham Banaepour Ghaffari - 518252005
Özcan Çelik - 518251039
Davide Colella - 912500295
Nurettin Özçelik - 518252003
Leonardo Russo - 912500322

May 22, 2026

Abstract

This project details the modeling, control optimization, and virtual validation of a 1:1 scale automated Anti-Aircraft Defense System (AADS) turret based on the OTO 76/62 SR naval platform. Designed to track agile targets like the MQ-9 Reaper UAV within a 5,000 meter range, the plant is modeled as a non-linear two-degree-of-freedom (2-DOF) serial mechanism using Euler-Lagrange dynamics coupled with a smooth Stribeck friction model to characterize bearing losses. To eliminate stick-slip limit cycles and dynamic cross-coupling, a bandwidth-separated cascaded control architecture is synthesized, dividing the system into decoupled inner current ($\omega_i = 600$ rad/s), middle velocity ($\omega_v = 60$ rad/s), and outer proportional position ($\omega_p = 5$ rad/s) loops featuring gravity and velocity feedforward compensation. Target state estimation and hardware latency protection are managed via a 9-state Extended Kalman Filter (EKF) utilizing a discrete white-noise jerk model and a numerically stable Joseph Form covariance update. The end-to-end perception pipeline fuses a K-band Metamaterial Electronically Scanned Array (MESA) radar for spatial cueing with a long-wave infrared (LWIR) payload and an RGB camera running a YOLO object detector (79.3% precision) verified by a Lucas-Kanade optical flow motion-consistency gate. Evaluated against a Simscape Multibody digital twin and tested in closed-loop within an NVIDIA Isaac Sim virtual war scenario, the system demonstrates high tracking fidelity, limiting steady-state root-mean-square (RMS) pointing errors below 0.2° with an average target miss distance of approximately 1 meter under max-capability threat maneuvers.

Contents

List of Figures

List of Tables

LIST OF SYMBOLS

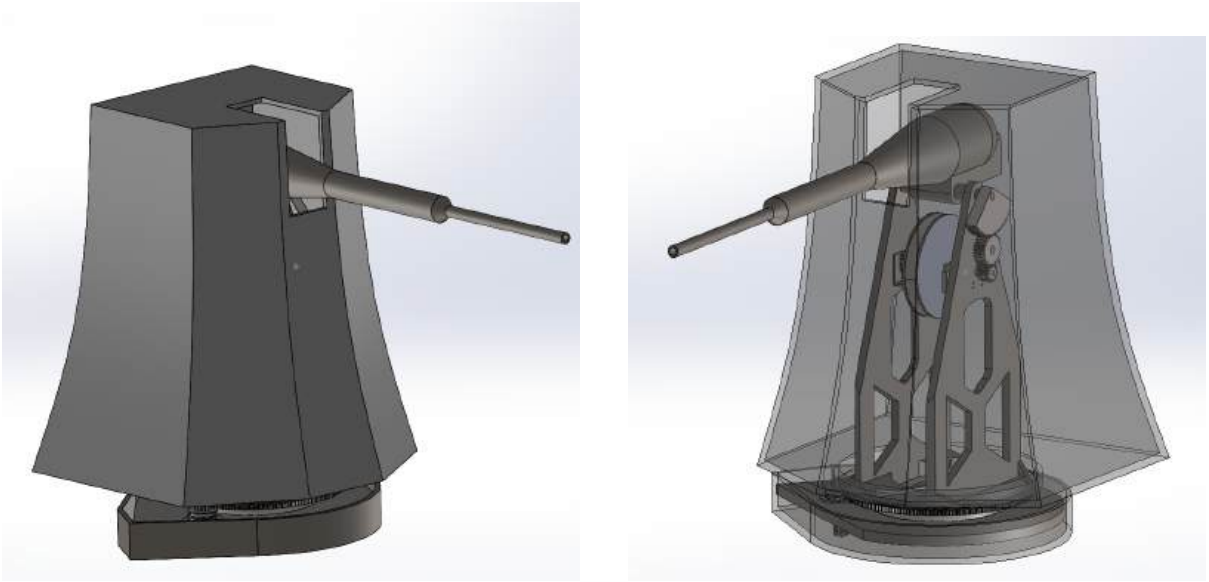
i_{az}	Azimuth transmission ratio
i_{el}	Elevation transmission ratio
$T_{\max}$	Maximum motor torque
T_{req}	Required axis-side torque
$T_{\mathrm{motor,req}}$	Required motor-side torque
ω_i	Current-loop bandwidth
ω_v	Velocity-loop bandwidth
ω_p	Position-loop bandwidth
ω_{motor}	Motor angular speed
η	Stage efficiency
η_{total}	Total drivetrain efficiency
σ_y	Yield strength

1 Mechanical Design and Transmission Architecture

1.1 Structure

The AADS gun mount is modelled at 1:1 scale after the OTO 76/62 SR naval artillery platform. The mechanical design prioritises a rigid load path from motor output to barrel trunnion while keeping the mass distribution close to the principal axis of inertia to minimise dynamic coupling between axes.

The assembly (Figures 1a and 1b) consists of three main structural groups: a fixed base carrying the azimuth slewing ring, a rotating turret body that encloses both drive trains, and an internal barrel cradle that pivots about the elevation trunnion axis. The truncated-pyramid turret cover provides structural rigidity and flat mounting surfaces for sensor payloads.



(a) AADS gun mount exterior (b) Internal layout (transparent cover)

Figure 1: AADS Gun Mount: Comparison of exterior and internal structural layout.

All gearing and structural components are manufactured from **AISI 17-4 PH** precipitation hardened stainless steel (H900/H1025 condition), selected for its high yield strength ($\sigma_y \geq 1170$), intrinsic corrosion resistance, and dimensional stability after heat treatment. The cover is a multilayer RAS, made of: Outer Skin (Unloaded E-Glass with Epoxy resin), Lossy Core Layers (CIP with Epoxy Resin), and Backing Skin (CFRP).

1.2 Azimuth and Elevation Gearboxes

Both axes employ the same architectural pattern: two cascaded epicycloidal (planetary) stages followed by a final spur output stage. A single actuator model, the **Bosch Rexroth MS2N07-E0BN** ($T_{\max} = 119.5$, $n_N = 3000rpm$, 24-bit absolute encoder), is used for both axes.

1.2.1 Actuator and Transmission Strategy

The motor nominal speed and the required output velocities set the minimum transmission ratios:

$$\omega_{\text{motor}} = \frac{2\pi \times 3000}{60} = 314.16 \tag{1}$$

$$i_{\text{min, az}} = \frac{314.16}{1.047} \approx 300 \qquad i_{\text{min, el}} = \frac{314.16}{0.611} \approx 514 \tag{2}$$

Design values of $i_{\text{az}} = 300$ and $i_{\text{el}} = 500$ are adopted, decomposed as:

$$i_{\text{az}} = 10 \times 10 \times 3 = 300 \tag{3}$$

$$i_{\text{el}} = 10 \times 10 \times 5 = 500 \tag{4}$$

With a per-stage efficiency of $\eta = 0.90$, the total driveline efficiency is $\eta_{\text{total}} = 0.9^3 = 0.729$. Table **??** confirms the motor is correctly sized for both axes.

Axis	i	T_{req} (N·m)	$T_{\text{motor,req}}$ (N·m)	Utilisation
Azimuth	300	16 447	75.2	62.9 %
Elevation	500	43 436	119.1	99.7 %

Table 1: Motor sizing verification — both axes

1.2.2 Azimuth Gearbox — 300:1

Planetary Stages 1 and 2. Both stages use a **fixed-ring epicycloidal** configuration with three distinct roles: *(i)* the **sun gear** acts as the rotational input (driven by the motor shaft in Stage 1, and by the Stage 1 planet carrier in Stage 2); *(ii)* the **ring gear** is rigidly fixed to the gearbox housing, providing the reaction member; and *(iii)* the **planet carrier** is the sole output, transmitting the amplified torque to the next stage. This configuration is preferred because fixing the ring gear yields the highest achievable ratio for a given tooth count, while the carrier output provides a naturally balanced load distribution across all planet meshes. Both stages share the tooth combination $Z_s = 18$, $Z_p = 72$, $Z_r = 162$, giving:

$$i_{1,2} = 1 + \frac{Z_r}{Z_s} = 1 + \frac{162}{18} = 10 \tag{5}$$

Stage 1 uses module $m = 3$ (face width 50); Stage 2 uses $m = 6$ (face width 120) to carry the tenfold torque increase while maintaining equivalent bending stress levels. Three planets are equally spaced at 120, satisfying the assembly condition $(Z_s + Z_r)/3 = 60 \in \mathbb{Z}$. Figure 2 shows the azimuth planetary gearbox sub-assembly.

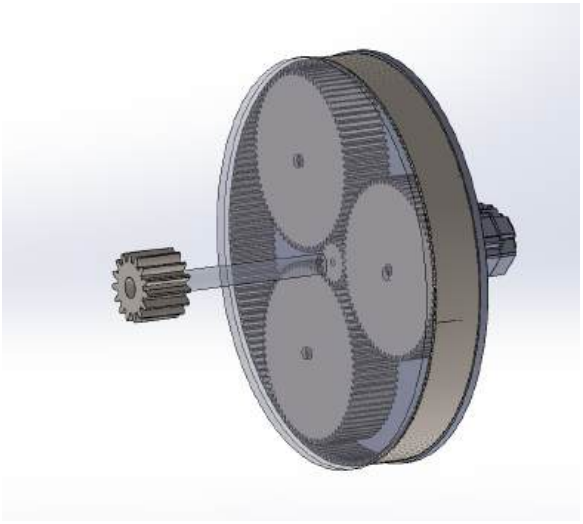


Figure 2: Elevation planetary sub-assembly

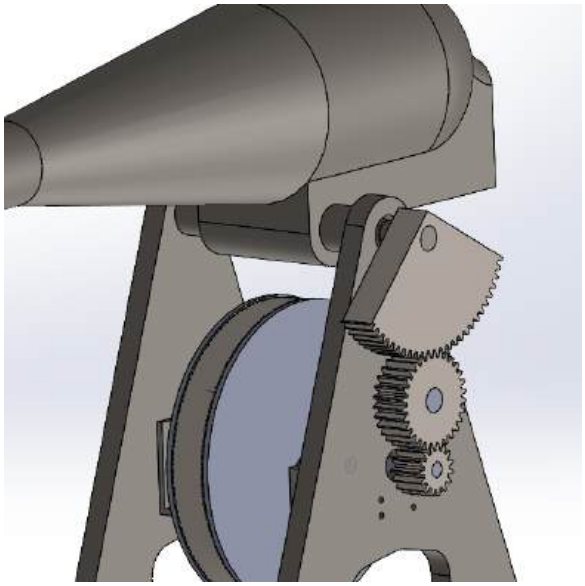


Figure 3: Elevation sector gear, 120° arc, $m = 10$, $Z_{\text{eq}} = 75$.

Final Stage — Pinion and Slewing Track. The Stage 2 carrier drives a pinion that meshes with a full 360 internal slewing ring (Figure 5), enabling continuous azimuth rotation. The stage ratio is $i_3 = 3$.

1.2.3 Elevation Gearbox — 500:1

Planetary Stages 1 and 2. The elevation planetary stages use the same **fixed-ring epicycloidal** configuration as the azimuth axis (sun input, fixed ring, carrier output), with identical tooth counts and module progression ($i_1 = i_2 = 10$, $m = 3 \rightarrow 6$). The sub-assembly geometry is identical to that shown in Figures 2 and 4.

Final Stage — Pinion, Idler, and Sector Gear. The final stage uses three $m = 10$ spur gears: drive pinion ($Z = 15$, $d = 150$), idler ($Z = 30$, $d = 300$), and a 120 sector ($Z = 75$, $d = 750$). The stage ratio is:

$$i_3 = \frac{Z_{\text{sector}}}{Z_{\text{pinion}}} = \frac{75}{15} = 5 \tag{6}$$

Centre distances between the three gears:

$$a_{12} = \frac{10(15 + 30)}{2} = 225 \tag{7}$$

$$a_{23} = \frac{10(30 + 75)}{2} = 525 \tag{8}$$

The idler bridges the 750 separation between the planetary output shaft and the barrel trunnion axis, and ensures the sector rotates in the same direction as the pinion. The 120 sector arc defines the usable elevation range of ±60.

1.2.4 Torque Flow and Parameter Summary

Table 2 consolidates all gear parameters and the computed output torque at each stage for both axes.

Axis	**Stage**	m	Z	i	T_{out} (N·m)
Azimuth	1 — Planetary	3	18/72/162	10	1 075.5
	2 — Planetary	6	18/72/162	10	9 679.5
	3 — Track	10	—	3	26 134.7
Elevation	1 — Planetary	3	18/72/162	10	1 075.5
	2 — Planetary	6	18/72/162	10	9 679.5
	3 — Sector	10	15/30/75	5	43 557.8
Azimuth total		—	—	**300**	**26 135**
Elevation total		—	—	**500**	**43 558**

Table 2: Complete gearbox parameter summary, both axes

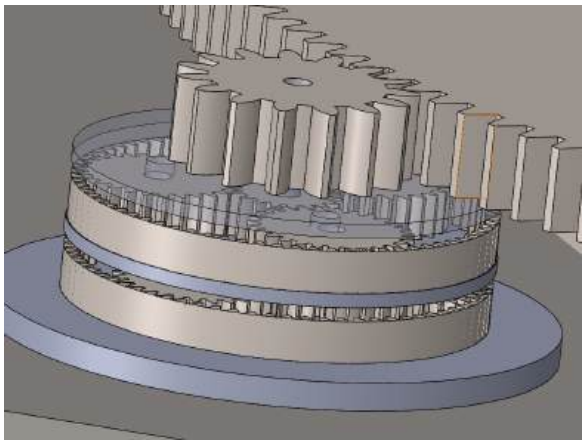


Figure 4: Azimuth planetary gearbox

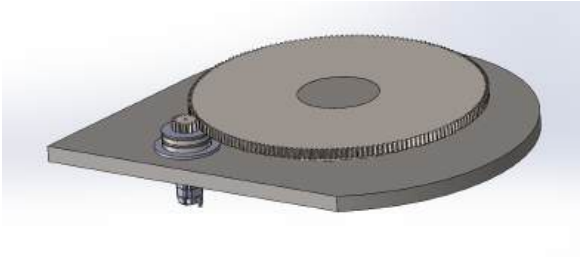


Figure 5: Azimuth slewing ring at the turret base

1.3 Actuation and Power Electronics

The primary actuators for the system are **Rexroth MS2N07-E0BN** synchronous AC servo motors (Permanent Magnet Synchronous Motors, PMSM). These units were selected for their high power density and optimized electromagnetic design, which facilitates superior dynamic response in high-precision tracking applications.

1.3.1 Actuator Specifications

The MS2N07-E0BN Rexroth motor provides a peak power of approximately 37.5 kW and a maximum torque (M_{Max}) of 119.5 Nm. When configured with forced water cooling, the continuous torque ($M_{0,Water}$) reaches 83.0 Nm.

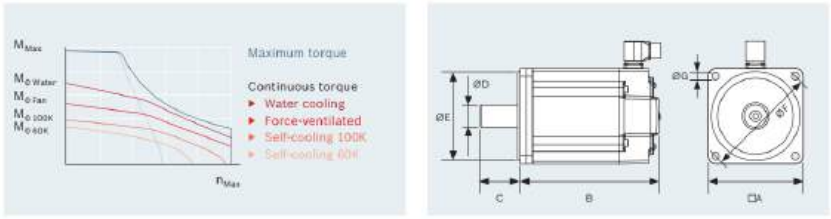


Figure 6: Torque-speed characteristic curves showing field weakening operation (Left) and MS2N mechanical dimensions (Right) .

Parameter	Value
Rotor Inertia (J_{rot}, without brake)	0.00331 kgm^2
Rotor Inertia (J_{rot}, with brake)	0.00355 kgm^2 (Estimated)
Rated Speed (n_N)	3,000 rpm
Max Speed (n_{Max})	6,000 rpm
Mass (Base Model)	12.0 kg
Mass (With Holding Brake)	14.0 kg
Shaft Diameter (D)	24 mm (k6)

Table 3: MS2N07-E0BN Mechanical parameters

1.3.2 Feedback sensor

Position feedback is managed by an integrated high-performance encoder system. The motor is equipped with a **24-bit digital absolute multi-turn encoder** (High performance level), providing a resolution of 2^{24} $(16,777,216)$ positions per revolution.

- **Resolution:** 24-bit via the digital ACURO®link interface.
- **Safety:** The encoder supports Functional Safety up to **SIL3 PLe**.
- **Absolute Sensing:** The multi-turn design tracks up to 4,096 revolutions, eliminating the need for homing cycles upon system restart.

2 Sensor Suite

To effectively address the stringent detection, tracking, and classification requirements of modern counter-UAS operations, the Autonomous Anti-Drone System (AADS) leverages a robust, multi-modal Sensor Suite. Relying on a single sensing modality leaves a system vulnerable to environmental blind spots—such as low-light conditions, heavy fog, or low-radar-cross-section targets.

2.1 3D MESA Radar

The primary sensor is the Echodyne EchoGuard, a software-defined MESA radar. It operates in the K-Band (24.05-24.25 GHz) and detects micro-UAS targets at ranges > 1.5 km.

Figure 7: Mesa radar implementation.

Specification	**Value / Description**
Core Technology	Metamaterial Electronically Scanned Array (MESA)
Frequency Band	24.05 – 24.25 GHz (K-Band)
Detection Range	$>$ 1.5 km (Micro-UAS / Drones) | $<$ 5km (big UAV)
Field of View (FOV)	120° Azimuth × 80° Elevation
Update Rate	Up to 15 Hz (3D Volume Scan)

Table 4: Mesa radar specification table

2.2 Thermal Optical Payload

Visual identification is provided by the Teledyne FLIR Boson LWIR camera. The 640×512 resolution core captures signatures in the 8 to 14 μm band at 60 Hz.

Figure 8: IR camera thermal sensor.

Specification	**Value / Description**
Detector Type	Uncooled VOx Microbolometer (LWIR)
Resolution & Pitch	640×512 pixels | 12 μm pixel size
Spectral Band	8 to 14 μm (Long-Wave Infrared)
Sensitivity (NETD)	$<$ 40 mK (High-contrast thermal isolation)
Frame Rate	60 Hz (Smooth tracking of fast targets)

Table 5: IR detector specification table

2.3 RGB Camera

The Sony FCB-EW9500H is a professional 4MP block camera featuring a high-sensitivity 1/1.8-type STARVIS™ CMOS image sensor with an enhanced 30× optical zoom. Designed for demanding imaging environments, the camera delivers excellent image quality, high dynamic range, and stable performance across a wide range of lighting conditions. With support for 4M resolution up to 60p

Figure 9: RGB Camera

Specification	Value / Description
Image Sensor	1/1.8-type STARVIS™ CMOS (Approx. 4.17M pixels)
Zoom	30x Optical (Enhanced), 36x StableZoom, 12x Digital
Horizontal Viewing Angle	58.1° (Wide) to 2.3° (Tele)
Minimum Object Distance	100 mm (Wide) to 1200 mm (Tele)
Zoom Speed (Wide to Tele)	2.9 s (Focus Tracking OFF) / 4.8 s (Focus Tracking ON)

Table 6: RGB Camera specification table

2.4 Angular servomotor sensor

The angular feedback is provided by a high-resolution absolute encoder integrated within the Rexroth MS2N07-E0BN motor housing. By utilizing the internal feedback architecture, the system achieves a compact form factor while maintaining high signal integrity for the control loop.

- **Resolution:** The system supports up to 24-bit digital resolution via the ACURO®link interface, providing 2^{24} distinct positions per revolution.
- **Absolute Feedback:** The multi-turn absolute design (4,096 revolutions) eliminates the requirement for a homing procedure upon system startup.
- **Safety Compliance:** The feedback unit is certified for Functional Safety up to SIL3 PLe, ensuring robust operation in safety-critical applications.

3 Tracking Error and Prediction

This section details the mechanics of the `EKFPredictor`, a 9-state Extended Kalman Filter (EKF) designed for high-performance 3D positional tracking and lead prediction. By maintaining higher-order kinematic states, the system forecasts future target locations, compensating for mechanical lag in servo-mounted turrets and control systems. An Extended Kalman Filter (EKF) was implemented for trajectory prediction. During testing with a 3-frame (100 ms) lead, prediction errors remained within acceptable limits:

- **RMS Azimuth (X) Error**: 9.34 mRad
- **RMS Elevation (Y) Error**: 12.10 mRad
- **RMS Range (Z) Error**: 4.46 m

3.1 Overview

The `EKFPredictor` ingests noisy sensor observations in real-time. Instead of purely filtering the current position, it estimates the true current position, velocity, and acceleration. This approach enables the calculation of lead prediction, essential for dynamic target tracking.

3.2 Core Mathematical Model

3.2.1 State Vector

The filter operates on a 9-dimensional state space containing the position (p), velocity (v), and acceleration (a) for all three spatial axes (X, Y, Z):

$$\mathbf{x} = \begin{bmatrix} x & y & z & v_x & v_y & v_z & a_x & a_y & a_z \end{bmatrix}^T \tag{9}$$

3.2.2 Motion Model (State Transition Matrix F)

The filter relies on a **Constant Acceleration Kinematic Model**. During the `PREDICT` step, the state is projected forward by the time-step Δt using standard kinematic equations:

$$\mathbf{p}_{k|k-1} = \mathbf{p}_{k-1|k-1} + \mathbf{v}_{k-1|k-1}\Delta t + \frac{1}{2}\mathbf{a}_{k-1|k-1}\Delta t^2 \tag{10}$$

$$\mathbf{v}_{k|k-1} = \mathbf{v}_{k-1|k-1} + \mathbf{a}_{k-1|k-1}\Delta t \tag{11}$$

$$\mathbf{a}_{k|k-1} = \mathbf{a}_{k-1|k-1} \tag{12}$$

3.2.3 Observation Model (H)

The system only directly observes the first 3 states (Position). The measurement Jacobian matrix $\mathbf{H}$ maps the state space to the observation space:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{13}$$

3.3 Noise & Uncertainty Modeling

3.3.1 Process Noise (Q) - Continuous White-Noise Jerk

Acceleration is rarely strictly constant in real-world scenarios. The `EKFPredictor` introduces uncertainty via the Process Noise matrix **Q**, specifically using a **Discrete White-Noise Jerk** model. It assumes that over any time step Δt, a random "jerk" (derivative of acceleration) occurs. The **Q** matrix is formulated by integrating this random jerk variance over time, properly correlating the uncertainty across position, velocity, and acceleration.

3.3.2 Adaptive Measurement Noise (R)

Sensors occasionally produce aggressive spikes or glitches. To mitigate this, the predictor uses **Innovation-Based Adaptive Measurement Noise**:

1. **Innovation:** Calculate the residual (difference between incoming measurement and expected target location).
2. **Windowing:** Maintain a rolling buffer of innovation magnitudes (e.g., $N = 15$ frames).
3. **Scaling:** If the average innovation exceeds expected bounds, the base measurement noise matrix **R** is scaled up dynamically (e.g., up to $8.0\times$).

By inflating **R**, the filter temporarily trusts its internal kinematics over erratic incoming data.

3.4 Lead Prediction

After the Kalman `UPDATE` phase, the filter holds the optimal estimation of the target's current kinematics. To output the lead prediction, this state is projected exactly N_{lead} frames into the future using the constant-acceleration model. This generates an exact position for the turret's PID loop to aim toward.

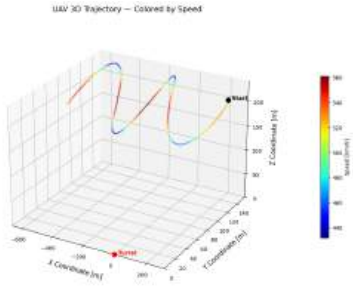


Figure 10: UAV worst case trajectory profile.

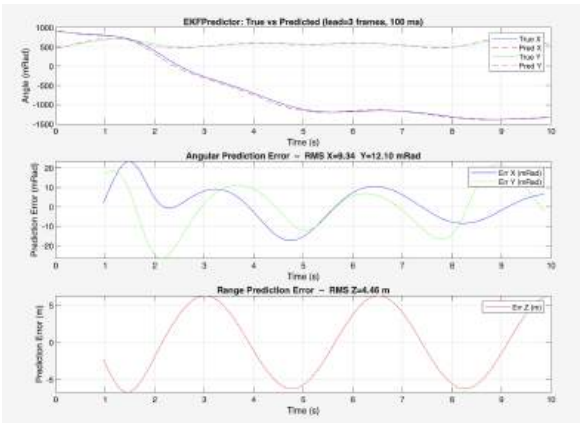


Figure 11: Prediction Results.

4 Mathematical Modeling

4.1 Rigid-Body Load Dynamics

The turret is modeled as a two-degree-of-freedom rigid-body system with azimuth yaw motion and elevation pitch motion. The Euler-Lagrange method is used to derive the load-side equations of motion because it captures inertia variation and coupling between the two axes.

The generalized coordinates describing the system state are defined as:

$$q = \begin{bmatrix} \theta \\ \alpha \end{bmatrix} \tag{14}$$

where θ denotes the azimuth angle and α denotes the elevation angle relative to the horizontal plane.

Instead of using a full 3D inertia tensor for the complete assembly, the load inertia is separated according to the azimuth and elevation coordinates. This also avoids double-counting the motor and gearbox inertias, which are added later through the reflected inertia model.

The inertia of the system is characterized by three principal parameters:

- J_{base}: The constant rotational inertia of the azimuth structure, representing components that rotate about the vertical axis but do not articulate in elevation.
- J_{barrel}: The rotational inertia of the elevating mass about the horizontal pitch axis.
- J_b: The maximum azimuthal inertia contribution of the elevation assembly, evaluated when the barrel is horizontal ($\alpha = 0$).

The model assumes that the rotation axes are aligned with the main inertia axes, so products of inertia are neglected. The total kinetic energy of the load side is the sum of the azimuth and elevation kinetic energies:

$$K = \frac{1}{2}\left[J_{base} + J_b \cos^2(\alpha)\right]\dot{\theta}^2 + \frac{1}{2}J_{barrel}\dot{\alpha}^2. \tag{15}$$

The term $J_b \cos^2(\alpha)$ dictates that the azimuth inertia decreases as the barrel elevates, reaching a minimum at $\alpha = \pi/2$. Therefore, the azimuth dynamics depend on the elevation position.

The potential energy arises solely from gravitational forces acting on the elevation assembly. Defining m as the mass of the elevating structure, r as the distance from the elevation axis to the center of gravity along the barrel, and d as the orthogonal vertical offset, the potential energy is:

$$P = mg\big(r\sin\alpha + d\cos\alpha\big). \tag{16}$$

The system Lagrangian, $L = K - P$, is thus:

$$L = \frac{1}{2}\left[J_{base} + J_b \cos^2(\alpha)\right]\dot{\theta}^2 + \frac{1}{2}J_{barrel}\dot{\alpha}^2 - mg\big(r\sin\alpha + d\cos\alpha\big). \tag{17}$$

Applying the standard Euler-Lagrange operator, $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i} = \tau_{j,i}$, yields the rigid-body torques required at the joints.

$$\tau_{j,\theta} = \left[J_{base} + J_b \cos^2(\alpha)\right] \ddot{\theta} - J_b \sin(2\alpha)\dot{\alpha}\dot{\theta}. \tag{18}$$

$$\tau_{j,\alpha} = J_{barrel}\ddot{\alpha} + \frac{1}{2} J_b \sin(2\alpha)\dot{\theta}^2 + mgr\cos(\alpha) - mgd\sin(\alpha). \tag{19}$$

These scalar equations of motion can be assembled into the standard matrix formulation:

$$M_L(q)\ddot{q} + C_L(q,\dot{q})\dot{q} + G(q) = \tau_j - \tau_{dist} \tag{20}$$

where τ_j is the applied joint torque vector, and τ_{dist} encapsulates external disturbances. The respective load-side inertia, Coriolis, and gravity matrices are explicitly defined as:

$$M_L(q) = \begin{bmatrix} J_{base} + J_b\cos^2(\alpha) & 0 \\ 0 & J_{barrel} \end{bmatrix} \tag{21}$$

$$C_L(q,\dot{q}) = \begin{bmatrix} -\frac{1}{2}J_b\sin(2\alpha)\dot{\alpha} & -\frac{1}{2}J_b\sin(2\alpha)\dot{\theta} \\ \frac{1}{2}J_b\sin(2\alpha)\dot{\theta} & 0 \end{bmatrix} \tag{22}$$

$$G(q) = \begin{bmatrix} 0 \\ mgr\cos(\alpha) - mgd\sin(\alpha) \end{bmatrix} \tag{23}$$

4.2 Gear Transmission and Reflected Inertia Modeling

The rigid-body model in Section 4.1 describes only the load-side turret dynamics. To complete the mechanical model, the motor rotor, gearbox inertia, and viscous losses are reflected to the joint side through the gear ratios. Assuming an infinitely stiff drivetrain with zero backlash, the kinematic relationship between the motor shaft velocity, $\omega_{m,i}$, and the joint angular velocity, $\dot{q}_i$, is strictly proportional:

$$\omega_{m,i} = N_i \dot{q}_i, \tag{24}$$

where N_i denotes the gear ratio for axis $i \in \{\theta, \alpha\}$, defined as the ratio of motor speed to joint speed.

For the two-axis system, this relation becomes

$$\boldsymbol{\omega}_m = \boldsymbol{N}\dot{\boldsymbol{q}}, \qquad \boldsymbol{N} = \begin{bmatrix} N_\theta & 0 \\ 0 & N_\alpha \end{bmatrix}. \tag{25}$$

The motor and gearbox rotating parts are not included in the CAD-based load inertia $\boldsymbol{M}_L(\boldsymbol{q})$. Their kinetic energy is modeled at the motor shaft as

$$K_{m,i} = \frac{1}{2} J_{m,i}^{eq} \omega_{m,i}^2, \tag{26}$$

where $J_{m,i}^{eq}$ represents the equivalent lumped inertia at the motor shaft. This parameter aggregates the inertias of the motor rotor, the primary pinion, and any intermediate gearbox components referred back to the high-speed shaft.

Substituting $\omega_{m,i} = N_i\dot{q}_i$ gives:

$$K_{m,i} = \frac{1}{2}\left(N_i^2 J_{m,i}^{eq}\right)\dot{q}_i^2. \tag{27}$$

This indicates that the actuator inertia, when viewed from the low-speed joint side, is amplified by the square of the gear ratio. We define this reflected inertia as $J_{r,i} = N_i^2 J_{m,i}^{eq}$.

The two-axis reflected inertia matrix is

$$\boldsymbol{J}_r = \begin{bmatrix} N_\theta^2 J_{m,\theta}^{eq} & 0 \\ 0 & N_\alpha^2 J_{m,\alpha}^{eq} \end{bmatrix}. \tag{28}$$

The reflected inertias are added to the corresponding diagonal terms of the load inertia matrix:

$$\boldsymbol{M}(\boldsymbol{q}) = \boldsymbol{M}_L(\boldsymbol{q}) + \boldsymbol{J}_r = \begin{bmatrix} J_{base} + J_b \cos^2(\alpha) + J_{r,\theta} & 0 \\ 0 & J_{barrel} + J_{r,\alpha} \end{bmatrix}. \tag{29}$$

The elevation-dependent azimuth inertia term $J_b \cos^2(\alpha)$ remains part of the load-side dynamics.

Energy dissipation within the motors and high-speed bearings is modeled via linear viscous damping. Let $b_{m,i}$ denote the viscous friction coefficient at the motor shaft. Applying the same kinematic scaling principle used for inertia, the reflected joint-side viscous damping is:

$$B_{r,i} = N_i^2 b_{m,i}. \tag{30}$$

Thus, $B_r \dot{q}$ represents reflected linear actuator damping, while $F_L(\dot{q})$ represents nonlinear load-side friction.

The electromagnetic torque generated by the motor, $\tau_{m,i}$, is proportional to the armature current i_i via the torque constant $K_{t,i}$. The actual torque delivered to the turret joint must account for the mechanical advantage of the gear ratio and the inherent frictional losses of the transmission stage, represented by the efficiency factor η_i:

$$\tau_{j,i} = \eta_i N_i \tau_{m,i} = \eta_i N_i K_{t,i} i_i. \tag{31}$$

Defining a lumped electromechanical gain matrix $\boldsymbol{K}_j$, the joint torque vector is $\boldsymbol{\tau}_j = \boldsymbol{K}_j \boldsymbol{i}$, where:

$$\boldsymbol{K}_j = \begin{bmatrix} \eta_\theta N_\theta K_{t,\theta} & 0 \\ 0 & \eta_\alpha N_\alpha K_{t,\alpha} \end{bmatrix}. \tag{32}$$

Substituting these terms into the load-side dynamics gives the complete mechanical subsystem:

$$\big(\boldsymbol{M}_L(\boldsymbol{q}) + \boldsymbol{J}_r\big)\ddot{\boldsymbol{q}} + \boldsymbol{C}_L(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{G}(\boldsymbol{q}) + \boldsymbol{F}_L(\dot{\boldsymbol{q}}) + \boldsymbol{B}_r\dot{\boldsymbol{q}} = \boldsymbol{K}_j\boldsymbol{i} - \boldsymbol{\tau}_{dist}. \tag{33}$$

The equivalent motor-side inertias $J_{m,\theta}^{eq}$ and $J_{m,\alpha}^{eq}$ are calculated from the gearbox geometry in the next subsection.

4.2.1 Gearbox Internal Geometry and Lumped Parameter Sizing

This subsection calculates the equivalent motor-side inertias $J_{m,\theta}^{eq}$ and $J_{m,\alpha}^{eq}$. The first two planetary stages are common to both axes, while the final stage differs for the azimuth and elevation mechanisms.

The first two stages are fixed-ring planetary gear trains. For each stage, $Z_s = 18$, $Z_r = 162$, and the carrier is the output. Therefore,

$$i_{1,2} = 1 + \frac{Z_r}{Z_s} = 1 + \frac{162}{18} = 10. \tag{34}$$

Since the two planetary stages are cascaded, their component speeds relative to the motor shaft are

$$\omega_{sun,1} = \omega_m, \qquad \omega_{carrier,1} = 0.1\omega_m, \qquad \omega_{p,1} = -0.125\omega_m, \tag{35}$$

$$\omega_{sun,2} = 0.1\omega_m, \qquad \omega_{carrier,2} = 0.01\omega_m, \qquad \omega_{p,2} = -0.0125\omega_m. \tag{36}$$

The negative sign indicates opposite planet spin direction.

Gear components are modeled as homogeneous steel bodies with $\rho = 7850\,\mathrm{kg/m^3}$. For a solid cylindrical gear, the inertia and radius are given by $J = \frac{1}{2}mr^2$ and $r = \frac{m_{module}Z}{2}$, respectively.

Each planet gear has both spin and orbital motion. For $N_p = 3$ planets,

$$K_{planets} = N_p \left(\frac{1}{2} J_p \omega_p^2 + \frac{1}{2} m_p R_{orbit}^2 \omega_{carrier}^2 \right), \qquad R_{orbit} = r_{sun} + r_{planet}. \tag{37}$$

The equivalent motor-side reflected inertia of each planetary stage can be written in a common form:

$$\begin{aligned} J_{ref,k} = {} & J_{s,k} \left(\frac{\omega_{sun,k}}{\omega_m} \right)^2 + J_{c,k} \left(\frac{\omega_{carrier,k}}{\omega_m} \right)^2 \\ & + N_p \left[J_{p,k} \left(\frac{\omega_{p,k}}{\omega_m} \right)^2 + m_{p,k} R_{orbit,k}^2 \left(\frac{\omega_{carrier,k}}{\omega_m} \right)^2 \right], \qquad k = 1, 2. \end{aligned} \tag{38}$$

Azimuth Stage 3 Parametrization ($300:1$ **Total Reduction)** For the azimuth axis, the final stage gives $N_\theta = 300$ with $i_{3,\theta} = 3$. Carrier 2 drives the pinion, which meshes with the slewing ring:

$$\omega_{pinion,\theta} = 0.01\omega_m, \qquad \omega_{ring,\theta} = \frac{1}{300}\omega_m. \tag{39}$$

The slewing ring is modeled as a hollow ring, with inertia given by $J_{ring} = \frac{1}{2} m_{ring} \left(r_{out}^2 + r_{in}^2 \right)$.

Therefore, the azimuth equivalent motor-side inertia is

$$J_{m,\theta}^{eq} = J_m + J_{ref,1} + J_{ref,2} + J_{pinion,\theta} \left(\frac{\omega_{pinion,\theta}}{\omega_m} \right)^2 + J_{ring,\theta} \left(\frac{\omega_{ring,\theta}}{\omega_m} \right)^2. \tag{40}$$

Elevation Stage 3 Parametrization ($500:1$ **Total Reduction)** For the elevation axis, the final stage gives $N_\alpha = 500$ with $i_{3,\alpha} = 5$. The pinion, idler, and sector gear speeds are

$$\omega_{pinion,\alpha} = 0.01\omega_m, \qquad \omega_{idler} = -0.005\omega_m, \qquad \omega_{sector} = 0.002\omega_m. \tag{41}$$

Since the sector gear covers 120°, its inertia is taken as one third of the full gear inertia:

$$J_{sector} = \frac{120^\circ}{360^\circ} J_{full} = \frac{1}{3} \left(\frac{1}{2} m_{full} r_3^2 \right). \tag{42}$$

The elevation equivalent motor-side inertia is then

$$J_{m,\alpha}^{eq} = J_m + J_{ref,1} + J_{ref,2} + J_{pinion,\alpha} \left(\frac{\omega_{pinion,\alpha}}{\omega_m} \right)^2 + J_{idler} \left(\frac{\omega_{idler}}{\omega_m} \right)^2 + J_{sector} \left(\frac{\omega_{sector}}{\omega_m} \right)^2. \tag{43}$$

After substituting the motor inertia J_m and gear geometry values, the model obtains the required $J_{m,\theta}^{eq}$ and $J_{m,\alpha}^{eq}$ values for simulation.

4.3 Actuator Electrical Dynamics Modeling

The mechanical model requires a torque-generation model for the actuators. Each servo motor is represented by an equivalent first-order armature model, assuming that the motor drive decouples flux and torque-producing current.

For each axis $i \in \{\theta, \alpha\}$, the electrical dynamics of the motor are governed by the standard armature circuit equation:

$$L_i \dot{i}_i + R_i i_i + K_{e,i} \omega_{m,i} = v_i, \tag{44}$$

where v_i is the applied terminal voltage (the control input), i_i is the armature current, L_i is the armature inductance, and R_i is the terminal resistance. The term $K_{e,i}\omega_{m,i}$ represents the back-electromotive force (back-EMF), which couples the mechanical velocity of the motor shaft back into the electrical domain.

Substituting the rigid transmission kinematic constraint, $\omega_{m,i} = N_i \dot{q}_i$, explicitly links the joint-space velocity to the internal electrical dynamics:

$$L_i \dot{i}_i + R_i i_i + K_{e,i} N_i \dot{q}_i = v_i. \tag{45}$$

In vector notation, the electrical subsystem for the 2-DOF turret is expressed as:

$$\boldsymbol{L}\dot{\boldsymbol{i}} + \boldsymbol{R}\boldsymbol{i} + \boldsymbol{K}_e \boldsymbol{N} \dot{\boldsymbol{q}} = \boldsymbol{v}, \tag{46}$$

where $\boldsymbol{L}$, $\boldsymbol{R}$, and $\boldsymbol{K}_e$ are diagonal matrices containing the respective inductances, resistances, and back-EMF constants for the azimuth and elevation axes.

4.4 Friction Modeling

Linear viscous damping represents motor-side and transmission losses, but the main turret bearings also introduce nonlinear low-speed friction. Therefore, a static Stribeck friction model is used on the load side. The friction terms are separated to avoid double-counting. The nonlinear model represents load-side bearing friction, while $\boldsymbol{B}_r \dot{\boldsymbol{q}}$ represents reflected motor and transmission damping. The model includes breakaway friction, Coulomb friction, and load-side viscous friction. Breakaway friction is larger than Coulomb friction, and the Stribeck term describes the decrease between these levels at low speed.

For each turret axis $i \in \{\theta, \alpha\}$, the total load-side friction torque is modeled as the sum of Stribeck, Coulomb, and viscous components. The ideal mathematical representation is:

$$\tau_{fL,i}(\dot{q}_i) = \left[\tau_{c,i} + (\tau_{brk,i} - \tau_{c,i}) \exp\left(-\frac{|\dot{q}_i|}{\omega_{s,i}} \right) \right] \operatorname{sgn}(\dot{q}_i) + b_{L,i} \dot{q}_i, \tag{47}$$

where $\tau_{brk,i}$ is the breakaway friction torque, $\tau_{c,i}$ is the Coulomb friction torque, $\omega_{s,i}$ dictates the Stribeck velocity threshold, and $b_{L,i}$ is the purely load-side viscous friction coefficient.

Physical constraints dictate that the breakaway torque must exceed or equal the Coulomb torque ($\tau_{brk,i} \geq \tau_{c,i}$). The sign function is discontinuous at zero velocity, which can cause numerical chattering during velocity reversals or position holding. Therefore, the sign function is replaced by the smooth approximation

$$\operatorname{sgn}(\dot{q}_i) \approx \tanh\left(\frac{\dot{q}_i}{\omega_{c,i}} \right), \tag{48}$$

where $\omega_{c,i} > 0$ defines the smoothing velocity constant.

The continuous friction model utilized for system simulation becomes:

$$\tau_{fL,i}(\dot{q}_i) = \left[\tau_{c,i} + (\tau_{brk,i} - \tau_{c,i}) \exp\left(-\frac{|\dot{q}_i|}{\omega_{s,i}}\right)\right] \tanh\left(\frac{\dot{q}_i}{\omega_{c,i}}\right) + b_{L,i}\dot{q}_i. \tag{49}$$

This form keeps the breakaway and Coulomb friction behavior while making the model continuous near zero velocity. The friction vector $\boldsymbol{\tau}_{fL}(\dot{\boldsymbol{q}})$, is added on the mechanical load side:

$$\big(\boldsymbol{M}_L(\boldsymbol{q}) + \boldsymbol{J}_r\big)\ddot{\boldsymbol{q}} + \boldsymbol{C}_L(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{G}(\boldsymbol{q}) + \boldsymbol{\tau}_{fL}(\dot{\boldsymbol{q}}) + \boldsymbol{B}_r\dot{\boldsymbol{q}} = \boldsymbol{K}_j\boldsymbol{i} - \boldsymbol{\tau}_{dist}. \tag{50}$$

4.5 External Torque Disturbance Modeling

In operation, the turret is affected by external loads such as wind gusts, recoil, and chassis vibration. These effects are difficult to model separately, so they are represented by a lumped disturbance torque vector τ_{dist}.

In standard multi-body dynamic formulations, exogenous forces acting on the mechanical structure are introduced as generalized torques. For a 2-DOF turret system comprising azimuth (θ) and elevation (α) axes, the rigid-body equation of motion takes the form:

$$M_L(q)\ddot{q} + C_L(q, \dot{q})\dot{q} + G(q) + F_L(\dot{q}) = \tau_j - \tau_{dist} \tag{51}$$

where τ_j represents the joint torque applied by the actuators, and τ_{dist} is the external torque disturbance vector.

The disturbance for each axis is modeled as a combination of a step input and a sinusoidal vibration term:

$$\tau_{dist,i}(t) = C_i u(t - t_0) + A_i \sin(2\pi f_{vib} t) \tag{52}$$

The step term $C_i u(t - t_0)$ represents sudden disturbances such as recoil or wind gusts. The sinusoidal term $A_i \sin(2\pi f_{vib} t)$ represents periodic disturbances such as chassis vibration or torque ripple.

4.6 Multibody system

To capture the complex dynamic interactions of the defense turret, a high-fidelity model was developed using **Simscape Multibody** within the Simulink environment. This approach allows for a realistic simulation by accounting for the physical properties imported from the CAD assembly, such as mass, centers of gravity, and inertia tensors.

The system is organized into two main hierarchical subsystems: the *Base and Azimuth group* and the *Gun Mount and Elevation group*. Figures 12 and 13 illustrate the internal block architecture of these components, highlighting the mechanical constraints and the reference frames used for the joints.

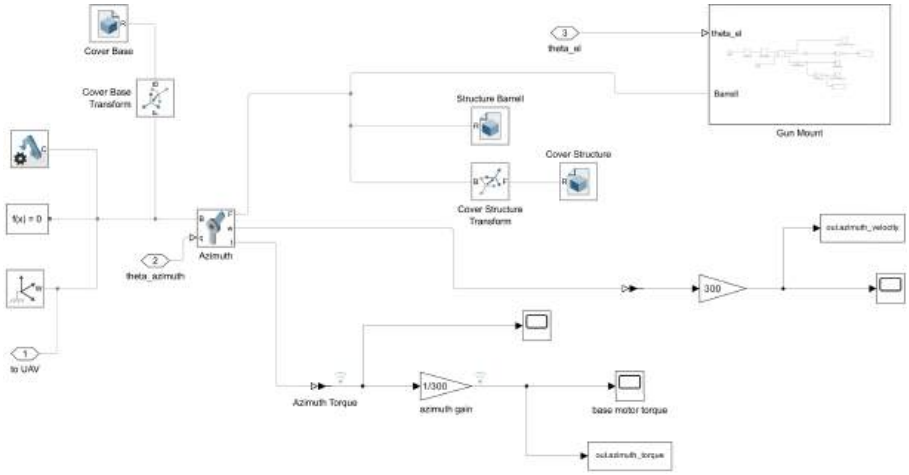


Figure 12: Simulink architecture of the Azimuth subsystem.

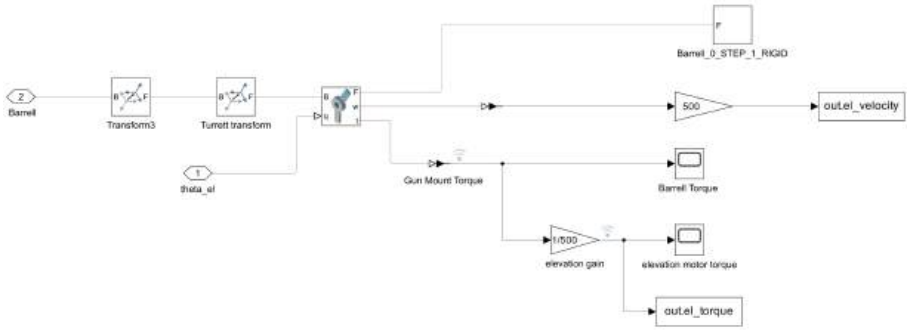


Figure 13: Simulink architecture of the Elevation subsystem.

The Multibody model receives as inputs the joint angles $\theta_{azimuth}$ and $\theta_{elevation}$, generated by the **Inverse Kinematics** block. To ensure the simulation adheres to the physical reality of the OTO 76/62 turret, these signals and their respective derivatives (velocity and acceleration) are processed through **Saturation blocks**. These blocks enforce the operational limits specified in the manufacturer's datasheet ($\pm 60°/s$ and $\pm 72°/s^2$ for Azimuth; $\pm 35°/s$ and $\pm 72°/s^2$ for Elevation), preventing the model from reaching non-physical states.

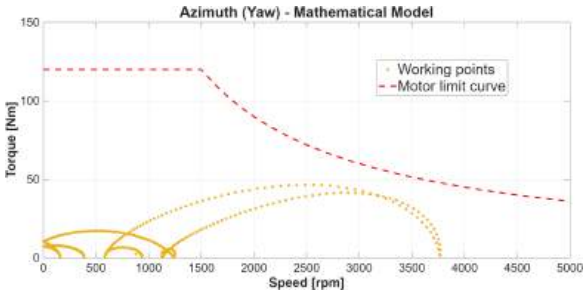
4.6.1 Mathematical model validation

In this framework, the **Simscape Multibody model acts as a "Digital Twin"** of the physical turret. Given that it incorporates the exact geometries and inertial properties from the CAD, as well as the constraints and kinematic saturations, it provides a high-fidelity benchmark that closely replicates the behavior of the real system.

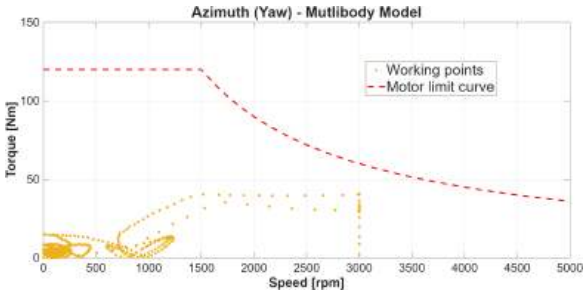
The validation was carried out by comparing the Torque-Speed ($T - \omega$) characteristic curves. Specific *To Workspace* blocks were integrated to export the motor velocities and torques from the Multibody environment, using the actual gear ratios ($N_{yaw} = 300$ and $N_{pitch} = 500$).

As illustrated in Figures 14 and 15, the torque profiles generated by the state-space model show an excellent correlation with the saturated Multibody simulation. Minor discrepancies are attributable to the inherent differences in how the two models handle

inertia and to the presence of strict kinematic limits in the Simscape blocks. However, the overall agreement validates the mathematical representation, confirming that the selected motors provide a sufficient safety margin and that the model is reliable for the development of the complex control architecture.

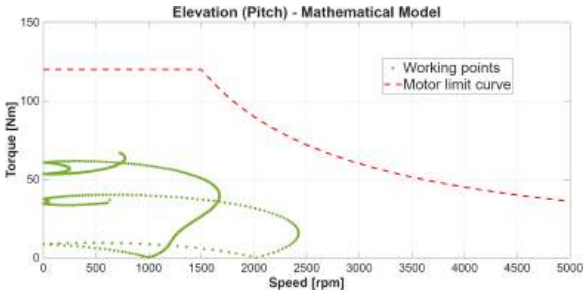


(a) State space representation

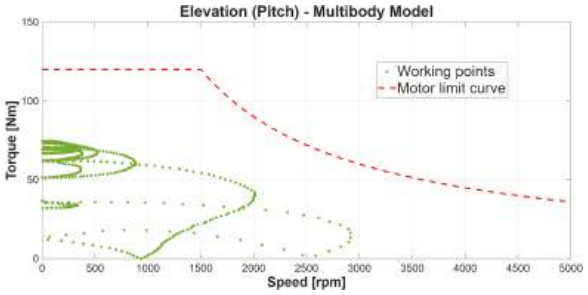


(b) Multibody model

Figure 14: Azimuth: Torque vs Motor Curve.



(a) State space representation



(b) Multibody model

Figure 15: Elevation: Torque vs Motor Curve.

5 Control Architecture

5.1 General Control Structure

The main objective of the controller is to rotate the 2-DOF turret so that the barrel points toward the predicted UAV target position. The controlled variables are the azimuth angle θ and the elevation angle α. The complete Simulink implementation of the proposed control system can be seen in Figure 16.

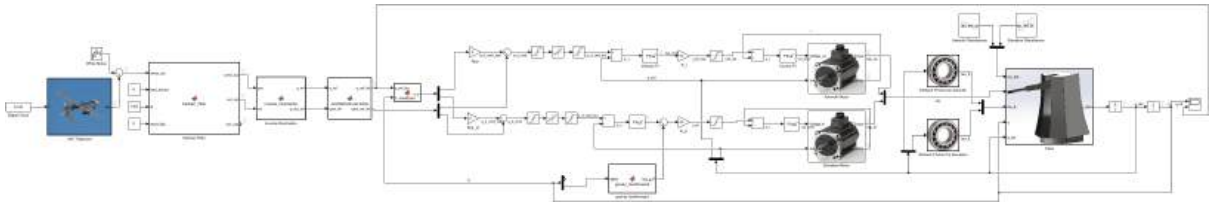


Figure 16: Overview of the complete Simulink model.

The target position and velocity are first converted into joint-space references by the inverse kinematics block. The resulting reference signals are

$$q_{ref} = \begin{bmatrix} \theta_{ref} \\ \alpha_{ref} \end{bmatrix}, \qquad \dot{q}_{ref} = \begin{bmatrix} \dot{\theta}_{ref} \\ \dot{\alpha}_{ref} \end{bmatrix}. \tag{53}$$

A cascaded control structure, which is shown in Figure 17, is used for both turret axes. This structure separates the control problem into three levels: position control, velocity control, and current control. The outer position loop generates a velocity command, the middle velocity loop generates a torque command, and the inner current loop generates the motor voltage command. The general signal flow is

$$q_{ref} \rightarrow e_p \rightarrow \dot{q}_{cmd} \rightarrow e_v \rightarrow \tau_{cmd} \rightarrow i_{ref} \rightarrow e_i \rightarrow v_{cmd} \tag{54}$$

This architecture is selected because the system contains dynamics with different time scales. The electrical current dynamics are the fastest part of the system. The mechanical velocity dynamics are slower because they depend on the large turret inertia, friction, and external disturbances. The position dynamics are the slowest because joint position is obtained by integrating joint velocity. By placing the fastest loop inside and the slowest loop outside, each loop can be tuned more clearly.

The same cascaded structure is used for the azimuth and elevation axes. However, the elevation axis also includes a gravity feedforward term because the barrel mass creates a configuration-dependent gravitational torque. This feedforward term reduces the amount of torque that must be produced only by the velocity-loop integrator.

Before the main control action, the joint references are passed through a reference limiter. The elevation angle is limited between 30° and 90°, while the azimuth axis is treated as continuous. After the position loop generates the velocity command, velocity and acceleration limits are applied. These limits prevent the controller from commanding motion that exceeds the physical capability of the turret.

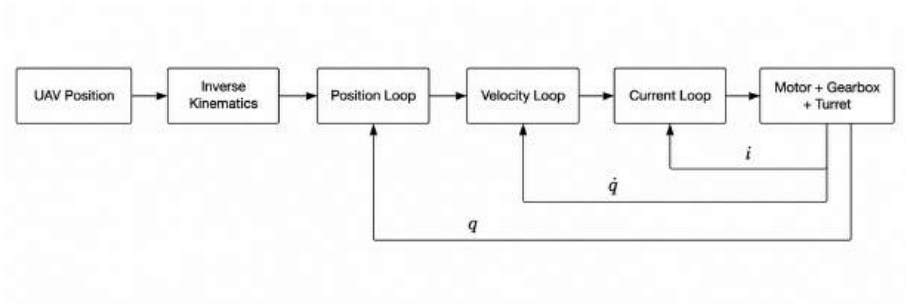


Figure 17: Overview of control architecture

5.1.1 Outer Position Loop

The outer position loop is responsible for reducing the angular pointing error of the turret. It receives the limited joint position reference $q_{ref,lim}$ and the measured joint position q. It generates the desired joint velocity command for the next control layer.

The position error is defined as

$$e_p(t) = q_{ref,lim}(t) - q(t). \tag{55}$$

For the azimuth axis, a direct subtraction can produce an incorrect large error when the reference crosses the $\pm\pi$ boundary. Therefore, the azimuth error is calculated by using the shortest angular distance:

$$e_\theta(t) = \operatorname{atan2}\left(\sin(\theta_{ref,lim}(t) - \theta(t)), \cos(\theta_{ref,lim}(t) - \theta(t))\right). \tag{56}$$

For the elevation axis, direct subtraction is sufficient because the elevation motion is limited between 30° and 90°:

$$e_\alpha(t) = \alpha_{ref,lim}(t) - \alpha(t). \tag{57}$$

A proportional controller is used in the outer loop. The velocity command is generated as

$$\dot{q}_{cmd,raw}(t) = K_{pp} e_p(t) + \dot{q}_{ref,lim}(t). \tag{58}$$

In this equation, $K_{pp} e_p(t)$ is the feedback part and $\dot{q}_{ref,lim}(t)$ is the velocity feedforward part. The feedback term corrects the pointing error, while the feedforward term helps the turret follow a moving target without a large continuous lag.

The use of a proportional controller is sufficient because the controlled plant from velocity command to position behaves approximately as an integrator when the inner velocity loop is fast. If the velocity loop tracks the command accurately, then

$$\dot{q}(t) \approx \dot{q}_{cmd}(t). \tag{59}$$

For a constant reference, this gives

$$\dot{e}_p(t) = -K_{pp} e_p(t). \tag{60}$$

Therefore, the position error follows a stable first-order response. Adding integral action in the position loop is not required because the velocity loop already contains integral action to reject friction and disturbance effects. A position integral term could also increase overshoot and interact with the velocity and acceleration limits. After the raw velocity command is generated, it is passed through velocity saturation and rate limiter blocks.

5.1.2 Middle Velocity Loop

The middle loop controls the joint angular velocity. It receives the limited velocity command from the outer position loop and compares it with the measured joint velocity. The velocity error is defined as

$$e_v(t) = \dot{q}_{cmd,lim}(t) - \dot{q}(t). \tag{61}$$

The output of the velocity loop is a joint torque command. A proportional-integral controller is used:

$$\tau_{PI}(t) = K_{p,v} e_v(t) + K_{i,v} \int e_v(t)\, dt. \tag{62}$$

The use of PI control is important in this loop because the mechanical plant is affected by large inertia, viscous damping, nonlinear friction, and external torque disturbances. If only proportional control was used, a nonzero velocity error would be needed to generate the torque required to overcome friction and disturbances. The integral term removes this steady velocity error by producing the required steady torque offset.

The velocity dynamics can be approximated by a first-order mechanical model:

$$J\ddot{q}(t) + B\dot{q}(t) = \tau_{cmd}(t) - \tau_{load}(t), \tag{63}$$

where J is the effective inertia, B is the effective viscous damping, and τ_{load} represents friction, gravity, coupling effects, and external disturbance torques. This simplified model shows why the velocity loop must generate torque, not position or voltage directly.

The azimuth and elevation axes use the same PI structure, but with different gains because their inertias and torque capacities are different. For the elevation axis, a gravity feedforward term is added after the velocity PI controller. This is required because the barrel mass creates a gravitational torque that changes with the elevation angle. The feedforward torque is

$$\tau_{g,ff}(t) = mgr\cos(\alpha(t)) - mgd\sin(\alpha(t)). \tag{64}$$

Therefore, the final elevation torque command is

$$\tau_{cmd,\alpha}(t) = \tau_{PI,\alpha}(t) + \tau_{g,ff}(t). \tag{65}$$

The torque commands are limited according to the available peak joint torque of each actuator. In the final model, the azimuth torque command is limited to approximately ± 26135 N m, and the elevation torque command is limited to approximately ± 43558 N m. Anti-windup clamping is used in the PI controllers so that the integral term does not continue to increase when the actuator is saturated.

5.1.3 Inner Current Loop

The inner current loop is the fastest loop in the cascaded controller. Its purpose is to make the motor current follow the current reference, because the generated joint torque is proportional to motor current.

The torque command from the velocity loop is converted into a current reference using the joint torque gain:

$$i_{ref}(t) = \frac{\tau_{cmd}(t)}{K_j}. \tag{66}$$

For the two axes, the joint torque gains are

$$K_{j,\theta} = 454.896 \text{ N m/A}, \qquad K_{j,\alpha} = 758.16 \text{ N m/A}. \tag{67}$$

The current reference is limited by the peak motor current:

$$i_{max} = 57.45 \text{ A}. \tag{68}$$

This limit protects the motor and also represents the available peak torque capacity. In the elevation axis, the gravity feedforward torque is added before this conversion. Therefore, the current limiter acts on the combined feedback and feedforward torque demand.

The current tracking error is

$$e_i(t) = i_{ref,lim}(t) - i(t). \tag{69}$$

A PI controller is used to generate the voltage command:

$$v_{cmd}(t) = K_{p,i} e_i(t) + K_{i,i} \int e_i(t)\, dt. \tag{70}$$

The voltage command is limited to the available DC bus voltage:

$$-500 \text{ V} \leq v_{cmd} \leq 500 \text{ V}. \tag{71}$$

Anti-windup clamping is used in the PI controller to prevent excessive integral accumulation during voltage saturation. The resulting voltage command is applied to the motor model described previously in Section 4.3.

5.2 Controller Tuning Methodology

The cascaded controller was tuned from the innermost loop to the outermost loop. This order is used because each outer loop depends on the closed-loop behavior of the loop inside it. The main tuning principle is bandwidth separation. The inner loop must be faster than the outer loop so that it can be approximated as a fast actuator from the viewpoint of the outer controller. In this project, the selected loop bandwidths are

$$\omega_i = 600 \text{ rad/s}, \qquad \omega_v = 60 \text{ rad/s}, \qquad \omega_p = 5 \text{ rad/s}. \tag{72}$$

Here, ω_i is the current-loop bandwidth, ω_v is the velocity-loop bandwidth, and ω_p is the position-loop bandwidth. This separation is important for stability and practical implementation. The current loop handles the fast electrical dynamics, the velocity loop handles inertia, damping, friction, and disturbances, and the position loop handles the slower pointing motion. This structure also makes the controller easier to tune, because each loop can be adjusted after the faster inner loop has already been closed. After analytical gain selection, the gains were checked in simulation using step responses. The final gains were selected to obtain fast tracking while avoiding actuator saturation, excessive overshoot, and stick-slip oscillation.

5.2.1 Inner Current Loop Tuning

The innermost loop governs the transient response of the motor armature. The electrical plant transfer function, mapping applied voltage $V(s)$ to resulting current $I(s)$, is defined as:

$$G_e(s) = \frac{1}{Ls + R}$$

Where $L = 0.0075$ H and $R = 0.455\ \Omega$.

To enforce a first-order closed-loop response, a PI controller ($C_i(s)$) was designed using pole-zero cancellation. By setting the controller zero to exactly cancel the plant's electrical pole

$$\frac{K_{i,i}}{K_{p,i}} = \frac{R}{L}$$

the closed-loop transfer function simplifies to:

$$T_i(s) = \frac{\frac{K_{p,i}}{L}}{s + \frac{K_{p,i}}{L}}$$

This formulation directly maps the proportional gain to the desired closed-loop bandwidth:

$$\omega_i = \frac{K_{p,i}}{L}$$

To provide a sufficiently fast foundation for the mechanical loops, an aggressive target bandwidth of

$$\omega_i = 600 \text{ rad/s}$$

was selected. This yielded the electrical tuning parameters:

$$K_{p,i} = L \cdot \omega_i = 0.0075 \cdot 600 = 4.5$$

$$K_{i,i} = R \cdot \omega_i = 0.455 \cdot 600 = 273$$

A simulated 10 A step response confirmed a rise time of approximately 4.26 ms with 0% overshoot, validating the electrical linearization.

5.2.2 Middle Velocity Loop Tuning

With the current loop closed, the velocity loop regulates the mechanical rigid-body dynamics. The simplified linear plant from commanded torque $\tau(s)$ to joint velocity $\dot{q}(s)$ is

$$G_m(s) = \frac{\dot{Q}(s)}{(s)} = \frac{1}{Js + B}. \tag{73}$$

where J is the total effective inertia and B is the total viscous damping.
Pole-zero cancellation is again used by selecting

$$\frac{K_{i,v}}{K_{p,v}} = \frac{B}{J}. \tag{74}$$

This gives the velocity-loop tuning relations

$$K_{p,v} = J\omega_v, \qquad K_{i,v} = B\omega_v. \tag{75}$$

To maintain bandwidth separation, the velocity loop was targeted one decade below the current loop:

$$\omega_v = 60 \text{ rad/s}. \tag{76}$$

The total azimuth inertia is configuration-dependent:

$$J_{az}(\alpha) = J_{base} + J_b \cos^2(\alpha) + J_{r,az}. \tag{77}$$

To guarantee stability across the operating range, the azimuth controller was tuned for the maximum possible inertia. This occurs when the elevation angle is

$$\alpha = 0^\circ. \tag{78}$$

The corresponding maximum azimuth inertia is

$$J_{az,max} \approx 15{,}838.6 \text{ kg m}^2. \tag{79}$$

The elevation inertia is constant and is given by

$$J_{el} \approx 10{,}508.4 \text{ kg m}^2. \tag{80}$$

Using the derived J and B values, the velocity-loop gains were calculated as

$$\begin{aligned} K_{p,v,az} &= 950{,}318, & K_{i,v,az} &= 30{,}000, \\ K_{p,v,el} &= 630{,}503, & K_{i,v,el} &= 80{,}400. \end{aligned} \tag{81}$$

Simulated 0.1 rad/s step responses demonstrated rise times of approximately 51–60 ms. The slight deviation from the theoretical linear target of approximately 36 ms was an expected result of the integrator winding up to overcome the nonlinear Stribeck breakaway friction at zero velocity. Simultaneous two-axis step testing confirmed that the selected 60 rad/s bandwidth provides sufficient dynamic stiffness against Coriolis and centripetal cross-coupling effects.

5.2.3 Outer Position Loop Tuning

After the velocity loop is closed, the plant from commanded velocity to joint position can be approximated as an integrator:

$$G_p(s) = \frac{Q(s)}{\dot{Q}_{cmd}(s)} = \frac{1}{s}. \tag{82}$$

For this reason, a proportional controller is sufficient for the position loop:

$$C_p(s) = K_{pp}. \tag{83}$$

The resulting closed-loop transfer function is

$$\frac{Q(s)}{Q_{ref}(s)} = \frac{K_{pp}}{s + K_{pp}}. \tag{84}$$

Therefore, the position-loop bandwidth is approximately

$$\omega_p = K_{pp}. \tag{85}$$

Initial tuning used a position-loop bandwidth of

$$\omega_p = 10 \text{ rad/s}, \qquad K_{pp} = 10. \tag{86}$$

However, step testing showed limit cycling due to stick-slip behavior. The position loop commanded velocity reversals faster than the inner velocity loop could overcome the static friction region. As a result, the turret tended to overshoot and reverse repeatedly near the target position.

To avoid this nonlinear interaction, the position loop was detuned to

$$\omega_p = 5 \text{ rad/s}. \tag{87}$$

The final proportional gains are

$$K_{pp,az} = K_{pp,el} = 5. \tag{88}$$

With the velocity-loop bandwidth of $\omega_v = 60$ rad/s, the final bandwidth separation between the velocity and position loops is

$$\frac{\omega_v}{\omega_p} = \frac{60}{5} = 12. \tag{89}$$

This separation gives the velocity loop enough time to follow the velocity command generated by the position loop. The final value of $K_{pp} = 5$ eliminated the stick-slip limit cycling and produced a smooth approach to the target with no overshoot in the position step response.

5.3 Controller Performance Evaluation

The dynamic performance of the tuned control architecture was evaluated against a continuous 20-second simulated Unmanned Aerial Vehicle (UAV) trajectory. To establish a rigorous and realistic assessment, the evaluation metrics were bifurcated into two distinct operational regimes: the target acquisition phase (handling large initial transient errors) and the steady-state tracking phase (maintaining continuous pointing accuracy).

5.3.1 Performance Metrics

The controller was evaluated by using trajectory-tracking metrics instead of classical step-response metrics. Since the UAV reference is continuously changing, classical steady-state error, rise time, and percent overshoot are not suitable as the main performance indicators. Therefore, the final evaluation uses RMS tracking error, peak tracking error, line-of-sight RMS pointing error, RMS miss distance, and an initial overshoot-like error.

The main tracking metrics are calculated after the initial acquisition transient. In this work, the metric evaluation starts at $t = 5\,\mathrm{s}$. This prevents the large initial lock-on error from dominating the continuous tracking results.

The RMS tracking error is used to represent the average tracking accuracy:

$$e_{\theta,RMS} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_\theta^2(t_i)}, \qquad e_{\alpha,RMS} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_\alpha^2(t_i)}. \tag{90}$$

The peak absolute tracking error is used to show the worst tracking deviation during the evaluated part of the trajectory:

$$e_{\theta,peak} = \max|e_\theta(t_i)|, \qquad e_{\alpha,peak} = \max|e_\alpha(t_i)|. \tag{91}$$

The combined line-of-sight pointing error is calculated from the azimuth and elevation errors:

$$e_{LOS}(t) = \sqrt{e_\theta^2(t) + e_\alpha^2(t)}. \tag{92}$$

Here, the angular errors are expressed in radians. The reported LOS RMS value is given in milliradians:

$$e_{LOS,RMS} = 1000\sqrt{\frac{1}{N}\sum_{i=1}^{N} e_{LOS}^2(t_i)}. \tag{93}$$

The RMS miss distance is used to express the angular pointing error as an approximate physical distance at the UAV range. The range between the turret and the UAV is

$$R(t) = \sqrt{X^2(t) + Y^2(t) + Z^2(t)}. \tag{94}$$

Using the small-angle approximation, the instantaneous miss distance is

$$d_{miss}(t) \approx R(t)e_{LOS}(t). \tag{95}$$

The RMS miss distance is then calculated as

$$d_{miss,RMS} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} d_{miss}^2(t_i)}. \tag{96}$$

This value does not represent a full ballistic miss distance. It only represents the physical pointing error caused by turret line-of-sight tracking error.

An initial overshoot-like error is also reported for each axis. This metric is defined as

$$e_{OS,j} = \max\left(0, \max_{i\in\mathcal{J}_j}\left(q_j(t_i) - q_{ref,j}(t_i)\right)\right), \qquad j \in \{\theta, \alpha\}. \tag{97}$$

Here, $\mathcal{J}_j$ is the selected initial evaluation window for axis j. This is not classical percentage overshoot; it only measures the maximum positive actual-minus-reference deviation during the initial acquisition phase.

5.3.2 Evaluation Procedure

The controller was evaluated using five different UAV trajectory tests. Therefore, the comparison shows how the same controller performs under different target motion profiles. Each trajectory was simulated for 20 s. The controller performance was then evaluated by comparing the limited joint reference q_{ref} with the actual turret joint position q.

For each trajectory, the evaluation was based on three plots: the 3D UAV trajectory, the azimuth/elevation joint responses, and the azimuth/elevation position tracking errors. The 3D UAV trajectory plot shows the target motion used in the test. The joint response plot shows whether the turret follows the generated azimuth and elevation references. The tracking error plot shows the remaining angular error after the initial acquisition transient.

The numerical metrics were calculated from the same simulation data. The main tracking metrics were evaluated after $t = 5$ s, while the overshoot-like values were evaluated only inside selected initial time windows. This separation is used because the first part of the simulation represents target acquisition, while the later part represents continuous target tracking.

5.3.3 Trajectory Test Results

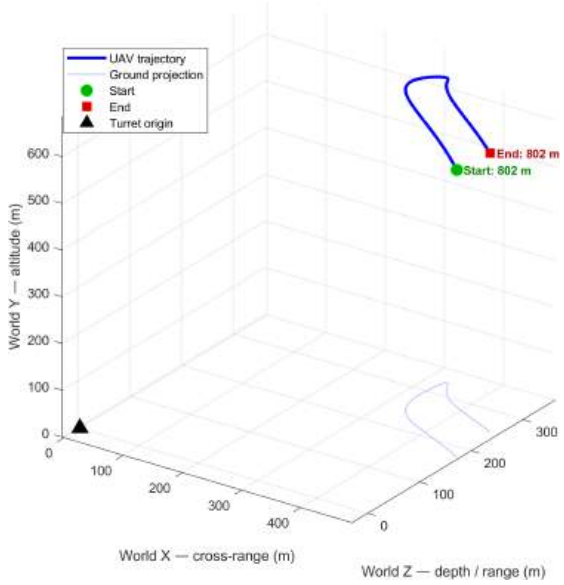
Five UAV trajectory cases were used to evaluate the same final controller under different target motion conditions. The controller gains, actuator limits, friction model, disturbance model, and reference limiting structure were kept unchanged for all tests. Therefore, the results show the robustness of the selected control structure against different target geometries and speed profiles.

The trajectory set was generated with a nominal target speed of approximately 50 m/s, a nominal range of 800 m, and a nominal elevation angle of 50°. The target coordinate convention is X for cross-range, Z for depth/range, and Y for altitude.

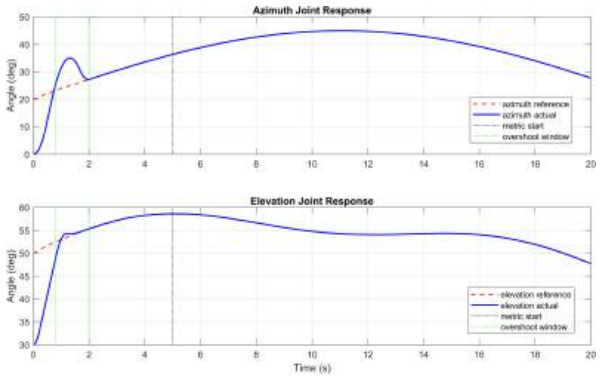
Table 7: Summary of UAV trajectory test cases

Trajectory	Motion type	Speed [km/h]	Altitude [m]	Range [m]
1	Smooth angular motion	11.2–180.0	597–687	802–803
2	Oscillatory crossing motion	138.6–179.6	585–641	758–955
3	Straight crossing motion	180.0	613	800–943
4	Weaving / evasive motion	28.8–180.0	573–653	820–890
5	Coordinated curving approach	24.5–180.0	416–589	664–800

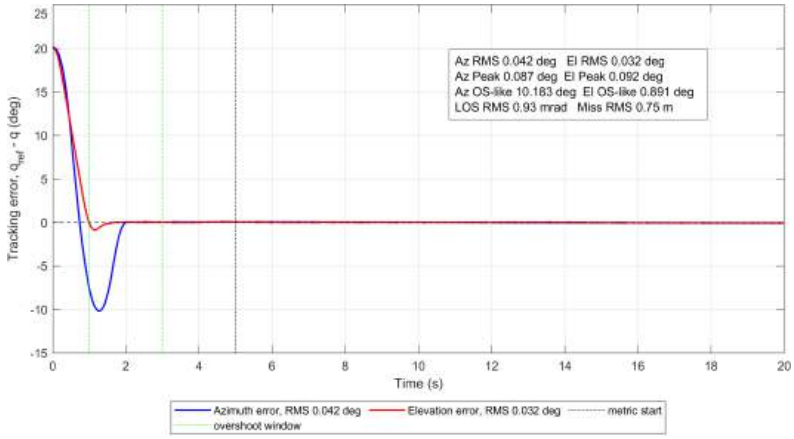
Trajectory 1 represents a smooth and joint limit-safe target motion. It is mainly used as the nominal tracking case because the range is almost constant and the reference changes smoothly.



(a) UAV trajectory



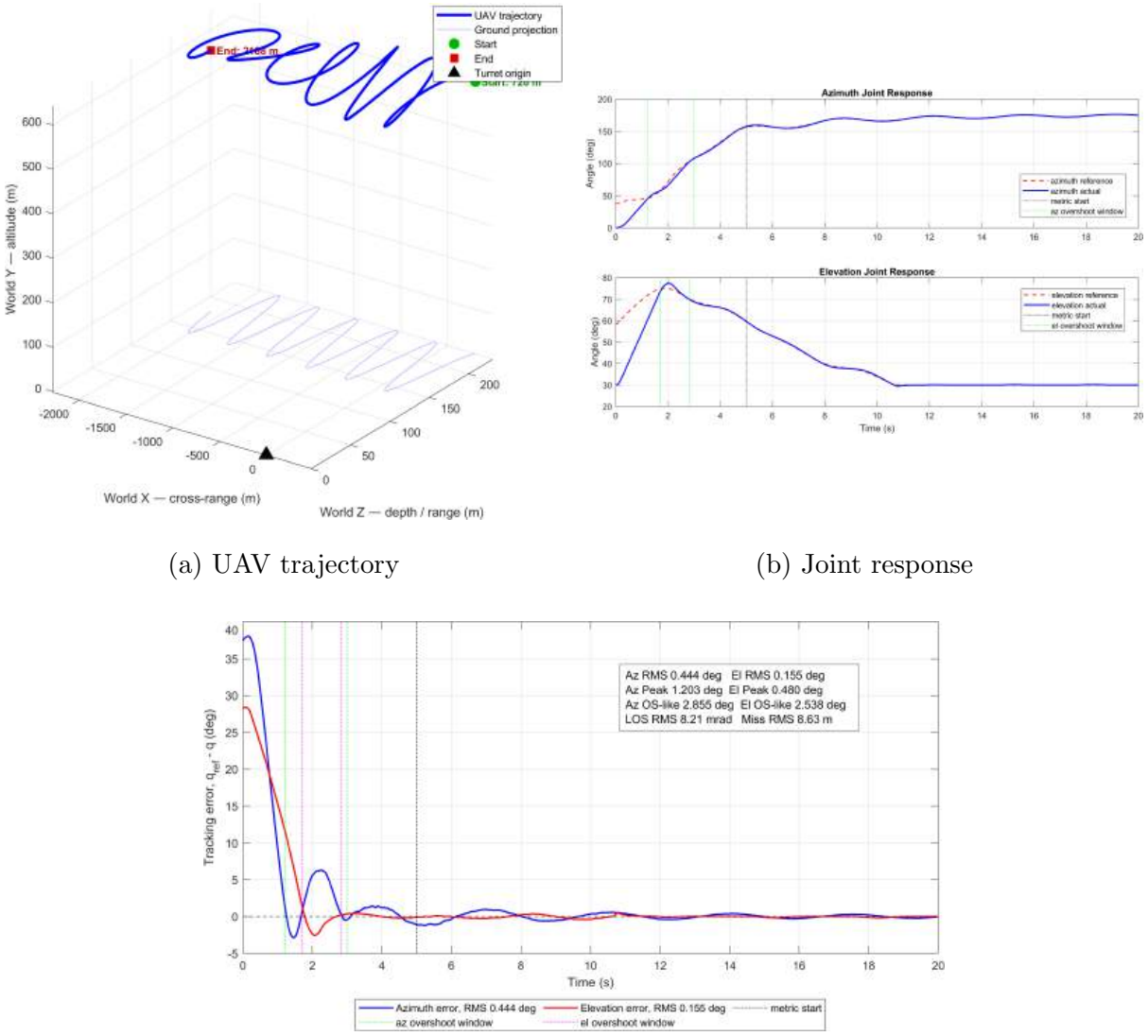
(b) Joint response



(c) Joint position tracking error

Figure 18: Controller performance results for Trajectory 1.

Trajectory 2 which is called "worst case", includes oscillatory motion in cross-range, depth, and altitude. This case produces faster reference variation and is useful for observing tracking delay and transient error.



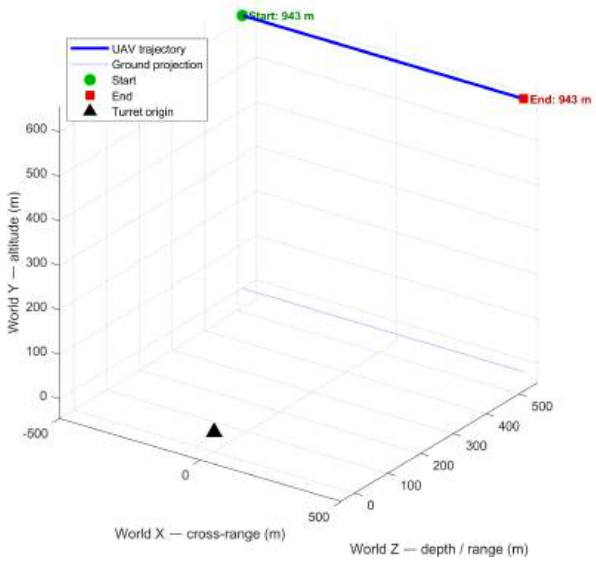
(a) UAV trajectory

(b) Joint response

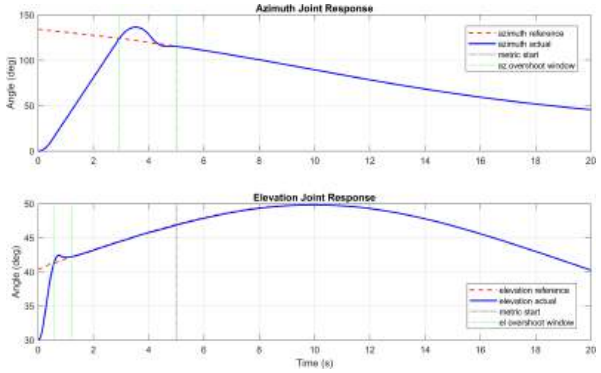
(c) Joint position tracking error

Figure 19: Controller performance results for Trajectory 2.

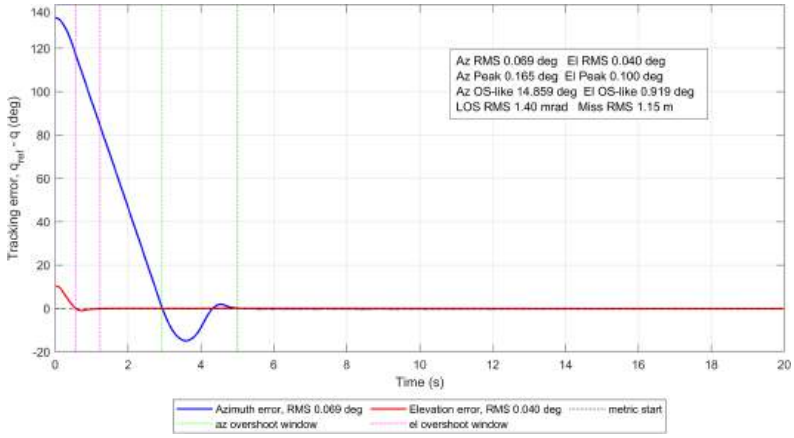
Trajectory 3 is a straight crossing case with constant speed and constant altitude. It tests the controller under a simple but continuous lateral target motion.



(a) UAV trajectory



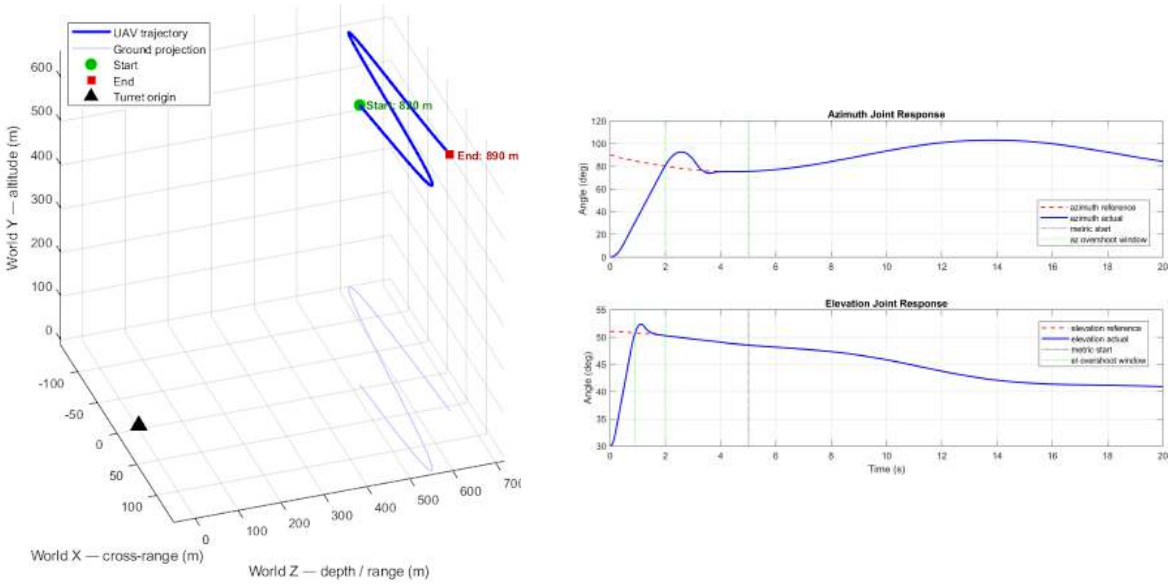
(b) Joint response



(c) Joint position tracking error

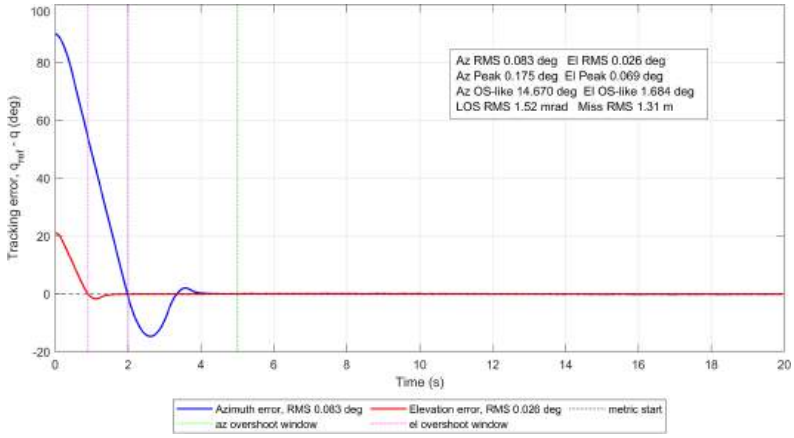
Figure 20: Controller performance results for Trajectory 3.

Trajectory 4 represents a weaving target. The target altitude and cross-range vary continuously, so this case is used to test the controller under evasive-like motion.



(a) UAV trajectory

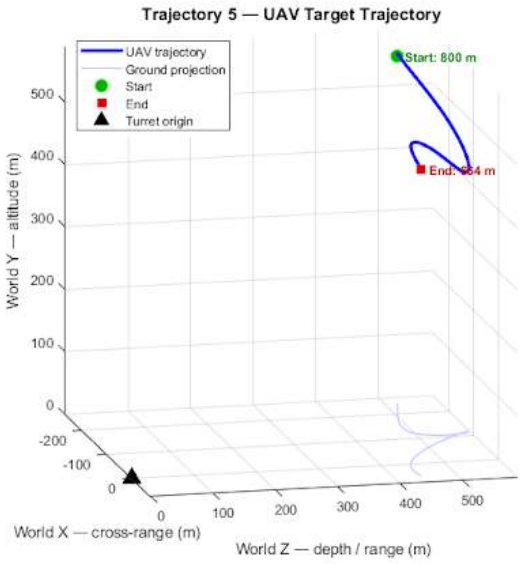
(b) Joint response



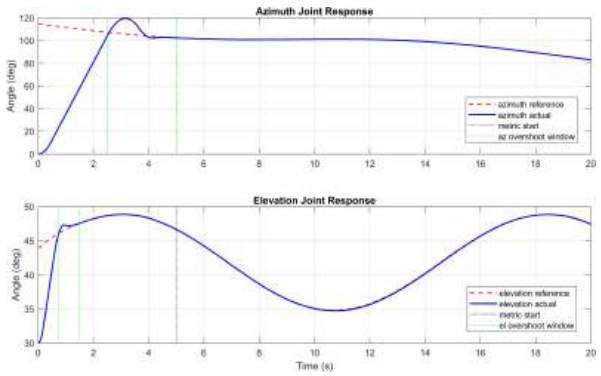
(c) Joint position tracking error

Figure 21: Controller performance results for Trajectory 4.

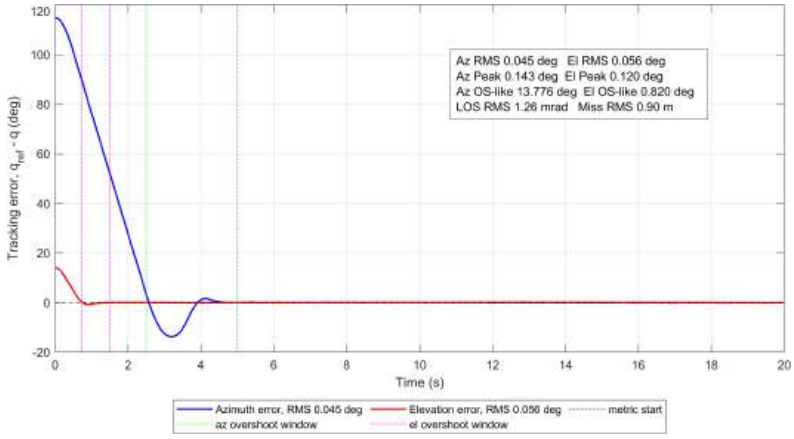
Trajectory 5 represents a coordinated curving approach. The target range decreases during the simulation, while azimuth and elevation also change. This case is useful for evaluating tracking performance during an approach-like motion.



(a) UAV trajectory



(b) Joint response



(c) Joint position tracking error

Figure 22: Controller performance results for Trajectory 5.

Table 8: Numerical controller performance results for the five UAV trajectory tests

Trajectory	Az RMS [deg]	El RMS [deg]	Az Peak [deg]	El Peak [deg]	Az OS-like [deg]	El OS-like [deg]	LOS RMS [mrad]	Miss RMS [m]
1	0.042	0.032	0.087	0.092	10.183	0.891	0.93	0.75
2	0.444	0.155	1.203	0.480	2.855	2.538	8.21	8.63
3	0.069	0.040	0.165	0.100	14.859	0.919	1.40	1.15
4	0.083	0.026	0.175	0.069	14.670	1.684	1.52	1.31
5	0.045	0.056	0.143	0.120	13.776	0.820	1.26	0.90

5.3.4 Discussion of Tracking Performance

The results show that the cascaded controller can track all five UAV trajectory cases without instability or sustained oscillation. In all cases, the largest errors occur during the initial acquisition phase. This is expected because the turret starts from its initial position, while the UAV reference may already require a different azimuth and elevation

angle. After the initial transient, the tracking errors decrease and remain small for most trajectories.

Trajectory 1 gives the best tracking result among the tested cases. This is expected because it is a smooth angular trajectory with nearly constant range. After the metric start time, the azimuth and elevation RMS errors are only 0.042° and 0.032°, respectively. The LOS RMS pointing error is 0.93 mrad, which corresponds to an RMS miss distance of approximately 0.75 m. This shows that the controller performs well for smooth nominal target motion.

Trajectory 2 is the most demanding case in the test set. It contains oscillatory motion in cross-range, depth, and altitude, and it produces faster changes in the reference angles. Therefore, it gives the largest tracking errors. The azimuth RMS error increases to 0.444°, while the elevation RMS error is 0.155°. The LOS RMS error is 8.21 mrad, and the RMS miss distance increases to 8.63 m. This larger miss distance is caused by both the larger angular tracking error and the relatively large UAV range in this trajectory. Even in this case, the response remains stable and the error does not diverge.

Trajectories 3, 4, and 5 show better continuous tracking performance than Trajectory 2. For these cases, the LOS RMS pointing error stays close to 1.3–1.5 mrad, and the RMS miss distance remains approximately between 0.90 m and 1.31 m. These results indicate that the controller can handle straight crossing, weaving, and curving approach motions with small residual tracking error after acquisition.

The initial overshoot-like error is larger in the azimuth axis for several trajectories. This does not mean that the controller has a large continuous tracking error. It mainly describes the initial lock-on behavior when the turret moves from its starting position toward the moving target reference. The elevation overshoot-like values are generally smaller, which indicates that the elevation axis reaches the reference more directly. The use of gravity feedforward also helps the elevation axis by reducing the torque that must be produced only by feedback action.

Overall, the final controller gives acceptable tracking accuracy for the five tested UAV trajectories. The nominal and moderate trajectories result in approximately meter-level RMS pointing miss distance. The worst-case oscillatory trajectory produces a larger error, but the system remains stable. Therefore, the results verify that the selected control structure is suitable for the nonlinear 2-DOF turret model, while also showing that more aggressive target motion may require a more advanced control technique for lower miss distance.

6 Isaac Sim virtual environment

NVIDIA Isaac Sim virtual environment has been used for its wide range of capabilities in multiphysics analysis, the main drive of our choice was the possibility of implementing radar sensors, as well as the availability of implementing machine learning training.

6.1 Simulation Objective

The objective of the Isaac Sim environment is to provide a realistic and repeatable test platform for the Anti-Aircraft Defense System (AADS) before real-world implementation. The simulation is used to evaluate how the defender observes, detects, tracks, and follows an airborne target under controlled conditions.

The MQ-9 UAV is defined as the primary target, while additional flying objects are included as distractors to create a more realistic multi-object scenario. This setup allows the perception and control pipeline to be tested under conditions where the system must distinguish the main target from other airborne objects.

Beyond visualization, Isaac Sim serves as an integration platform for the main sensing and control components of the project. The overall processing chain implemented in the simulation is shown in Figure 23.

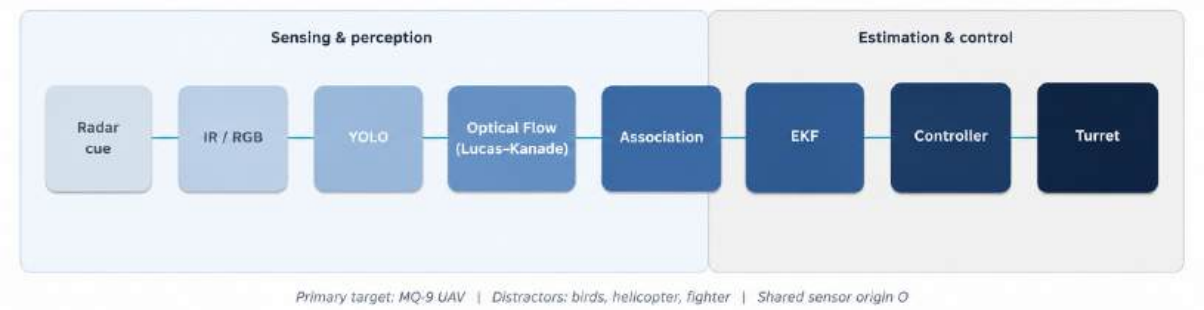


Figure 23: Integrated Isaac Sim pipeline for the AADS.

The pipeline begins with radar detection and RGB/IR image acquisition. YOLO provides object-level detections, while Lucas–Kanade optical flow is used to validate motion consistency. The confirmed target state is then estimated by the EKF and passed to the turret controller, which drives the defender platform toward the predicted aim point.

6.2 Virtual Scene and Environment Setup

The defender platform represents the Anti-Aircraft Defense System (AADS) and is placed at the world origin. This allows the platform to serve as the common reference point for radar, RGB camera, infrared camera measurements, sensor alignment, and turret pointing.

The MQ-9 Reaper UAV is used as the primary target, as shown in Figure 24. In the simulation, it follows a waypoint-based trajectory at kilometer-range distances, producing changing range, azimuth, and elevation values relative to the defender. Therefore, it is suitable for testing detection, tracking, sensor fusion, and turret response.

Figure 24: Isaac Sim virtual scene showing the defender platform.

Additional airborne objects were added as distractors to make the scene more realistic. These include bird groups, individual birds, a helicopter, and a distant F-16. Their purpose is to create visual clutter and test whether the perception system can distinguish the MQ-9 from other moving objects. The main modeled objects are summarized in Table 10.

Object / group	**Role**	**Purpose in simulation**
Defender platform	AADS unit	Sensor mounting, radar cueing, and turret motion
MQ-9 Reaper UAV	Primary target	Main object for detection, tracking, and engagement evaluation
Bird flocks	Distractors	Low-altitude local clutter near the defender
Bald eagle / black crow	Distractors	Small airborne distractors with different motion profiles
Ka-52 helicopter	Distractor	Medium-size aircraft near the engagement region
F-16 Fighting Falcon	Background distractor	Far-range airborne interference, not the primary engagement target

Table 9: Main objects modeled in the Isaac Sim environment.

6.3 Defender Platform, Target, and Distractor Models

The defender platform represents the Anti-Aircraft Defense System (AADS) and is positioned at the world origin. This placement allows the platform to act as the central reference for target observation, sensor alignment, and turret pointing. The sensor reference frame is attached to the defender structure so that radar, RGB camera, and infrared camera measurements can be generated from a common origin.

The MQ-9 Reaper UAV was selected as the primary target, as shown in Figure 25. In the simulation, the MQ-9 is modeled with an approximately realistic wingspan and follows a waypoint-based trajectory in the kilometer-range region. This motion profile produces continuous changes in range, azimuth, and elevation relative to the defender, making it suitable for testing target detection, tracking estimation, and turret response.

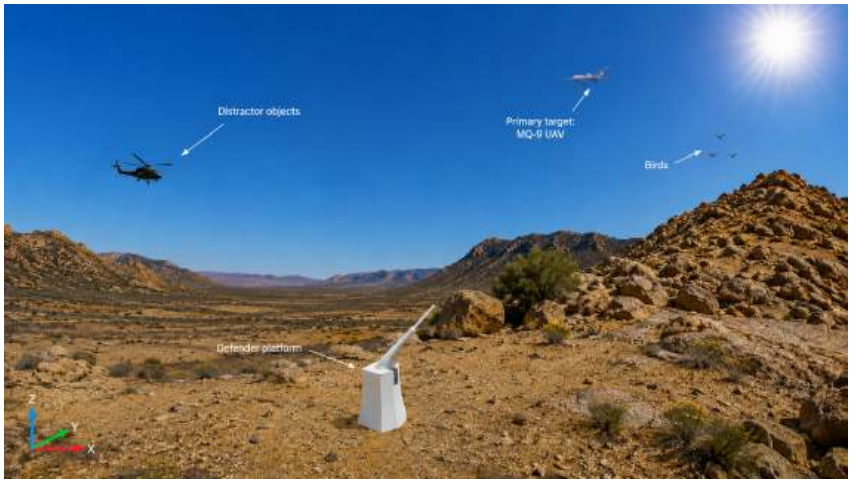


Figure 25: Isaac Sim virtual scene showing the defender platform, the MQ-9 UAV as the primary target, and additional airborne distractor objects including birds and a helicopter.

To avoid an overly idealized single-target scenario, additional airborne objects were included as distractors. These objects include birds and other aircraft-type models with different sizes, altitudes, and motion profiles. Their purpose is to create visual clutter and to test whether the perception pipeline can distinguish the primary MQ-9 target from other moving objects in the scene. Low-altitude distractors are useful for testing local clutter near the defender, while larger or faster aircraft models provide background airborne interference.

This modeling approach makes the simulation more representative of a real outdoor defense scenario. Instead of evaluating the AADS under isolated target conditions, the system is tested in a multi-object environment where target classification, motion consistency, and sensor fusion become necessary. The main modeled objects used in the Isaac Sim environment are summarized in Table 10.

Object / group	**Role**	**Purpose in simulation**
Defender platform	AADS unit	Sensor mounting, radar cueing, and turret motion
MQ-9 Reaper UAV	Primary target	Main object for detection, tracking, and engagement evaluation
Low-altitude distractors (6 scene groups, 15–120 m AGL)		
Bird flock (Bird1)	Distractor	Local clutter; waypoint patrol ∼30 m AGL
Bird flock (Bird2)	Distractor	Local clutter; waypoint patrol ∼32 m AGL
Bird flock (Bird3)	Distractor	Local clutter; waypoint patrol ∼35 m AGL
Bald eagle	Distractor	Soaring distractor; patrol ∼120 m AGL
Black crow	Distractor	Small avian distractor; patrol ∼35 m AGL
Ka-52 helicopter	Distractor	Medium-size aircraft near engagement region (∼40 m AGL)
Distant background		
F-16 Fighting Falcon	Background distractor	Far-range patrol (∼5 km AGL); not primary engagement focus

Table 10: Main objects modeled in the Isaac Sim environment.

6.4 Sensor Modeling and Data Acquisition

The Isaac Sim environment includes three sensing components mounted on the defender platform: a radar cueing model, an RGB camera, and an infrared (IR) camera. These sensors provide complementary information for target detection, visual confirmation, and tracking. A shared sensor reference frame was defined on the defender platform at an offset of [0, 0, 25] m from the defender origin, corresponding to the sensor deck location above the turret structure. This allows radar, RGB, and IR measurements to be expressed consistently in the same coordinate system. The common reference frame simplifies the conversion of sensor observations into target position estimates for the EKF and turret controller.

The main sensor parameters used in the simulation are summarized in Table 11.

Sensor	**Main Parameters**	**Role in Simulation**
Radar model	5000 m range, 14° beam width, K-band 24.05–24.25 GHz	Provides initial range, azimuth, and elevation cue for the target.
RGB camera	Native output up to 1920 × 1080 at 60 Hz; rendered at 640 × 512 in simulation	Provides visible images for YOLO-based detection and dataset generation.
Infrared camera	640 × 512, 60 Hz, 8–14 μm LWIR	Provides thermal observations for target confirmation and identification.
Shared origin	Located on defender sensor deck	Defines a common coordinate frame for radar, RGB, and IR outputs.

Table 11: Sensor parameters used in the Isaac Sim environment.

6.4.1 Radar Cueing

The radar model is used as the first stage of the sensing pipeline. Its role is to provide an initial cue for the airborne target by estimating the target range, azimuth, and elevation relative to the defender platform as shown in as shown in Figure 26. The radar model is based on a K-band sensing configuration, operating in the 24.05–24.25 GHz frequency range. In the simulation, the radar operates from the shared sensor origin and scans the engagement region using a defined beam width, range limit, and elevation coverage.

The radar cue is especially important because the camera sensors have a limited field of view compared with the full engagement space. Therefore, the radar output is used to guide the camera-based perception system toward the expected target direction. This reduces the search area for visual detection and supports the transition from wide-area surveillance to image-based target confirmation.

Figure 26: Isaac Sim radar telemetry view showing target detection, range estimation, azimuth and elevation angles, and contact information for the MQ-9 UAV and distractor objects.

6.4.2 RGB Camera

The RGB camera provides visible-spectrum image data for visual detection and dataset generation. In the simulation, the RGB camera is configured with a resolution of 640 × 512 pixels and a target frame rate of 60 Hz. The camera is mounted on the defender sensor deck and is aimed toward the target region during the simulation.

The RGB output is used mainly for YOLO-based object detection and bounding box generation. Since the simulation includes both the MQ-9 target and distractor objects, the RGB camera provides the image data required to evaluate whether the perception pipeline can distinguish the primary target from other airborne objects in the scene.

6.4.3 Infrared Camera

Real infrared cameras measure thermal radiation emitted by objects, mainly depending on temperature, emissivity, atmosphere, optics, and detector properties. In this project, the IR camera was modeled as an LWIR sensing channel inspired by the Teledyne FLIR Boson sensor, using an 8–14 μm spectral range, 640 × 512 image format, and 60 Hz target frame rate. A 73 mm lens with a 16 mm horizontal aperture was used to obtain a narrow field of view for long-range target observation.

The Isaac Sim implementation does not include a full radiometric heat-transfer model. Instead, a simplified thermal-appearance method was used to generate IR-like contrast for perception testing. Aircraft targets such as the MQ-9, Ka-52 helicopter, and F-16 were assigned high thermal intensity because their engines and exhaust regions are expected to dominate the infrared response. Bird distractors were assigned a warm biological signature, while terrain, defender structures, and background objects were treated as colder regions.

This approximation is sufficient for this simulation because the goal is not to estimate exact surface temperature, but to test whether infrared contrast can support target confirmation. The IR output was visualized using a Lava color map, where darker regions represent colder surfaces and brighter red, orange, and yellow regions represent higher apparent thermal intensity. The thermal view was generated in a separate IR rendering context, while the RGB cameras preserved their normal visible-spectrum appearance, as shown in Figure 27.

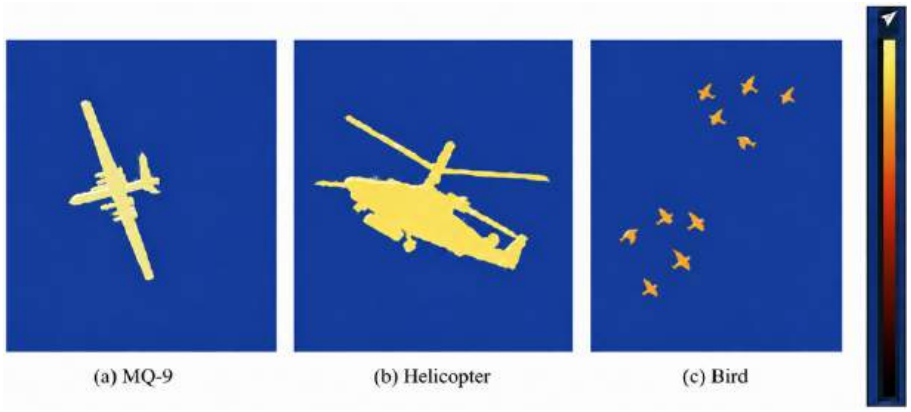


Figure 27: Thermal category visualization for representative airborne objects. From left to right: MQ-9, helicopter, and bird. Aircraft targets are assigned higher apparent thermal intensity than biological distractors.

The IR camera provides an additional sensing modality for confirming airborne targets when visible-spectrum detection alone may be insufficient. By combining RGB and IR observations, the simulation represents a more realistic multi-modal sensing pipeline for target detection, confirmation, and tracking.

6.5 Perception pipeline

The perception pipeline converts the raw sensor outputs from the Isaac Sim environment into target-level information that can be used by the tracking and control modules. RGB and infrared camera outputs are used for image-based detection, while radar cueing provides the initial target direction and range information. The pipeline combines object detection, motion consistency checking, and target association to identify the primary MQ-9 target in the presence of airborne distractors.

6.5.1 YOLO-Based Object Detection

YOLO-based object detection was used as the main image-based detection method in the perception pipeline. The objective of this stage is to convert RGB camera frames into object-level information, including bounding box location, object class, and confidence score. In the Isaac Sim environment, this allows the system to detect the MQ-9 target while also observing distractor objects such as birds, helicopters, and fighter aircraft. An example of the automatically generated annotations is shown in Figure 28. The bounding boxes illustrate how the MQ-9 target and distractor objects are labeled in the synthetic RGB images used for training.

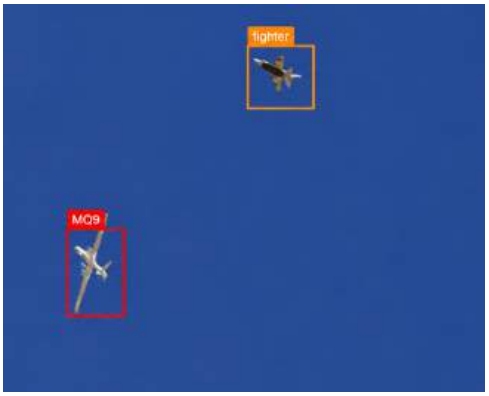


Figure 28: Example RGB frame from the Isaac Sim synthetic dataset with YOLO bounding boxes and class labels for the MQ-9 target and airborne distractor objects.

A synthetic dataset was generated directly from the Isaac Sim RGB camera outputs. Since the 3D simulation contains known object models, object positions, and semantic class labels, the corresponding bounding box annotations can be generated automatically during data collection. This avoids manual image labeling and ensures that each annotated frame is consistent with the simulated scene geometry. The dataset was collected using two RGB camera viewpoints, Camera 2 and Camera 3, in order to increase viewpoint diversity and reduce dependence on a single camera perspective.

The generated dataset contains 24,999 RGB images and 630,431 total bounding boxes. Among these frames, 8,134 images do not contain valid object annotations, corresponding to 32.54% of the collected dataset. These empty frames are useful for evaluating false-positive behavior, but they can also be filtered during training if a detector focused on positive target recognition is desired. The dataset statistics are summarized in Table 12.

Dataset Property	**Value**
Total RGB images	24,999
Total bounding boxes	630,431
Empty images	8,134
Empty image ratio	32.54%
Camera 2 images	14,999
Camera 3 images	10,000
Dataset format	YOLO object-detection format
Dataset split	80% training, 10% validation, 10% test

Table 12: Summary of the generated synthetic YOLO dataset.

The class map contains four airborne object categories: MQ-9, bird, helicopter, and fighter aircraft. The MQ-9 class represents the primary target, while the other classes represent distractors. This class separation is important because the perception system must not only detect airborne objects, but also distinguish the main target from clutter and non-target aircraft. The class distribution of the generated annotations is shown in Table 13.

Class	Number of Bounding Boxes
MQ-9	468,721
Bird	67,151
Helicopter	32,169
Fighter aircraft	62,390

Table 13: Class distribution of bounding box annotations in the synthetic dataset.

After data collection, the dataset was converted into the standard YOLO format. In this format, each image is paired with a text annotation file containing the class index and normalized bounding box coordinates. The processed dataset is organized into training, validation, and test subsets using an 80/10/10 split. This structure allows the model to be trained on the majority of the generated images, validated during training, and finally evaluated on a separate test subset.

During operation, the trained YOLO detector processes each RGB frame and returns candidate detections. Each detection includes a class label, a confidence score, and a bounding box in image coordinates. These outputs are then passed to the next stages of the perception pipeline, where motion consistency and sensor association are used to confirm whether the detected object corresponds to the primary MQ-9 target.

6.5.2 Optical Flow Motion Consistency Check

Object detection alone is not always sufficient to confirm a valid target, especially in a multi-object scene where birds, helicopters, and fighter aircraft may also appear in the camera field of view. For this reason, an optical-flow-based motion consistency check was added to the perception pipeline. Its purpose is to verify that the image motion observed in the RGB camera is consistent with the expected motion of a radar-cued contact and with the location of a YOLO detection.

The optical flow algorithm was implemented on the Camera 3 RGB viewport. Consecutive grayscale frames are extracted from the rendered camera output and processed inside a region of interest (ROI) centered near the expected target direction. The primary method is the Lucas–Kanade pyramidal optical flow algorithm. In each frame pair, corner features are first extracted using `goodFeaturesToTrack`, and then tracked using `calcOpticalFlowPyrLK`. The median displacement of the successfully tracked features is used as the estimated image motion.

The estimated pixel displacement is converted into angular correction using the camera focal length. This produces a small correction in milliradians, which can be compared with the radar-based sensor angles. A flow estimate is accepted only if enough valid features are found and if the residual motion inside the ROI remains below a predefined threshold. This prevents unstable flow values from being used when the image texture is weak or when the motion estimate is noisy.

In the perception pipeline, optical flow is used mainly as a validation stage rather than as the primary aiming source. The operational sequence is:

1. the radar provides an initial in-beam contact,
2. YOLO returns a candidate detection in the RGB image,
3. the radar-based aim direction is compared with the YOLO-based aim direction,

4. optical flow is used to check whether the observed image motion is consistent with the YOLO detection.

If the flow estimate is fresh and agrees with the YOLO aim within a tolerance of approximately 15°, the detection is accepted as motion-consistent. In the current configuration, a *soft validation mode* is used: if optical flow is unavailable or noisy, the system can still accept a YOLO detection when it agrees with the radar cue. This makes the pipeline more robust in low-texture or partially occluded conditions.

The main optical flow parameters used in the simulation are summarized in Table 14. In addition to online validation, the system logs both radar-only measurements (`meas_*`) and flow-assisted measurements (`flow_meas_*`) in the track-bus data stream. These signals can be compared offline in order to evaluate whether optical-flow-assisted measurements improve target-state estimation relative to radar-only measurements.

It should be noted that the implemented optical flow method is a classical 2D image-motion algorithm. It does not estimate full 3D target motion or depth directly. Its role in this project is therefore limited to motion consistency checking and small angular correction, not to replacing radar or EKF-based target-state estimation.

Parameter	**Description**
Camera source	Camera 3 RGB viewport
Primary algorithm	Lucas–Kanade pyramidal optical flow
Fallback algorithm	Farneback dense optical flow
ROI half-size	140 pixels
ROI search radius	120 pixels
Minimum valid features	4
Maximum residual motion	220 px/s
Maximum angular correction	25 mrad
YOLO–flow agreement tolerance	15°
Validation mode	Soft gate (radar + YOLO primary; flow used when available)

Table 14: Main parameters of the optical flow motion consistency algorithm.

6.5.3 Target Association and Confirmation

Target association and confirmation represent the final stage of the perception pipeline. At this stage, the system must decide which radar contact corresponds to a valid airborne target and which detections should be rejected as clutter or false alarms. This is especially important in the Isaac Sim scenario, where multiple moving objects are present at the same time, including the MQ-9 primary target, birds, a helicopter, and a fighter aircraft.

The association process begins with the radar contact list generated during each scan cycle. Each contact is defined by its name, world position, azimuth, elevation, and slant range relative to the shared sensor origin. When target handoff is enabled, the system selects an engaged contact from the in-beam radar detections. In the current configuration, the farthest in-beam contact with a range greater than or equal to 500 m is selected. This rule was chosen to reduce the influence of nearby low-altitude clutter, such as birds flying close to the defender platform, and to prioritize long-range contacts in the MQ-9 engagement band.

After a radar contact is selected, the system attempts to confirm it using the RGB perception outputs. The YOLO detector provides a candidate class label, confidence score, and bounding box location. The radar-based aim direction is then compared with the YOLO-based aim direction derived from the bounding box center and the radar slant range. Only detections belonging to accepted classes, currently `mq9` and `helo`, and exceeding the minimum confidence threshold are considered valid candidates.

Optical flow is used as an additional confirmation check when available. If a fresh optical-flow estimate exists and its residual motion is below the allowed limit, the flow-corrected aim direction is compared with the YOLO aim direction. If the two agree within approximately 15°, the contact is accepted as motion-consistent. In the implemented soft validation mode, a contact may still be accepted when YOLO agrees with the radar cue even if optical flow is missing or noisy. This improves robustness in low-texture image regions or during brief tracking interruptions.

If a contact fails validation repeatedly, it is rejected and added to a temporary rejected-contact list. After eight consecutive failed validation frames, the system switches to the next available in-beam contact. During a new radar scan, the rejected-contact list is cleared so that previously rejected objects may be reconsidered if they re-enter the beam under more favorable conditions. When the radar enters a `LOCKED` or `TRACK` state, the engaged contact may be retained even if it briefly leaves the beam, allowing the IR camera, EKF, and turret controller to continue following the same target.

Once a contact is confirmed, it becomes the active engaged target for the rest of the perception and control chain. The infrared camera is synchronized to this engaged contact instead of using timer-based switching between nearby objects. The confirmed target position is then passed to the EKF and turret controller for state estimation and pointing. In this way, target association and confirmation connect the radar cue, visual detection, motion validation, and closed-loop tracking modules into one integrated decision process.

6.6 Closed-Loop Integration

The final stage of the Isaac Sim pipeline connects the perception outputs to target-state estimation and turret motion. After a radar contact is selected and confirmed through YOLO and optical-flow validation, the system enters a closed-loop tracking mode. In this mode, radar measurements are converted into target-state estimates, an Extended Kalman Filter (EKF) is used for prediction, and a cascaded turret controller drives the defender platform toward the predicted aim point. This subsection describes how the EKF and turret controller are integrated within the simulation.

6.6.1 Extended Kalman Filter Integration

The EKF used in the Isaac Sim closed loop is the same 9-state constant-acceleration filter described in Section 3. It estimates the target position, velocity, and acceleration in the shared sensor coordinate frame at origin O. In the simulation, the filter receives slant-range radar measurements derived from the confirmed engaged target selected by the perception pipeline.

At each simulation step, the target position is converted from radar azimuth, elevation, and range into Cartesian coordinates relative to the sensor origin. These values are passed to the EKF as the measurement vector. The filter then performs prediction and update

steps using the constant-acceleration motion model. To compensate for sensor latency and mechanical tracking delay, a lead prediction of 2 frames is applied, corresponding to approximately 33 ms at the simulation rate of 60 Hz. This lead point represents the future target position toward which the turret should aim.

The EKF parameters used in the closed-loop simulation were tuned from Isaac replay data. In the current configuration, the process noise is set to 6000 and the measurement noise to 0.0001. An adaptive measurement-noise mechanism is also used to reduce the influence of occasional noisy radar updates. When target handoff switches to a new engaged contact, the EKF state is reset so that the filter starts from the newly confirmed target.

In the default closed-loop configuration, the turret uses the radar-based aim point directly, while the EKF output is logged as `ekf_pred_*`. Alternatively, the aim source can be switched to `ekf_lead`, in which case the turret follows the EKF lead prediction instead of the raw radar measurement. This allows the effect of predictive tracking to be evaluated directly inside Isaac Sim. If the EKF output is temporarily invalid, the system falls back to radar-based aiming.

6.6.2 Turret Control Integration

The turret controller converts the selected aim point into azimuth and elevation commands for the defender platform. In the simulation, the turret is implemented using a split-axis motion model: azimuth rotation is applied to the defender base, while elevation rotation is applied to the barrel pivot. This structure matches the mechanical architecture described in the system model and allows independent control of the two turret axes.

The controller is implemented as a cascaded position–velocity control law ported from the Simulink model. The outer loop generates a desired joint velocity from the angular error between the current turret orientation and the commanded aim direction. The inner loop applies a velocity PI controller with anti-windup and axis-specific torque limits. The controller gains and plant parameters, including gear ratios of 300:1 for azimuth and 500:1 for elevation, were matched to the analytical turret model developed in Section 4.

In Isaac Sim, the controller operates in kinematic mode. Instead of applying motor torques directly to a full electromechanical plant, the controller output is converted into joint reference angles, which are then used to drive the turret motion. This approach is suitable for evaluating pointing performance, tracking error, and closed-loop behavior within the virtual environment. When tracking begins, the controller state is synchronized to the current aim reference to avoid a large initial transient.

The turret motion is subject to operational rate limits defined in the simulation configuration. The azimuth axis is limited to approximately 60 deg/s and the elevation axis to 35 deg/s. Continuous azimuth rotation is also enabled so that the turret can follow targets across the full 360° range without angle wrapping artifacts. During active tracking, the controller receives the aim point selected by the closed-loop chain: During active tracking, the closed-loop chain operates as follows:

1. radar or confirmed-target measurement,
2. optional EKF lead prediction,
3. turret aim point generation,
4. joint reference computation,

5. defender platform motion.

this way, the perception, estimation, and control modules are integrated into one closed-loop anti-aircraft tracking system within Isaac Sim.

7 Simulation Results

7.1 Object Detection and Optical-Flow Validation

The detector was trained using YOLOv8n (`yolov8n.pt`) for 20 epochs (run `train-6`, batch size 4, image size 640). The final validation metrics are summarized in Table 15, and the training curves are shown in Figure 29. The results show relatively high precision but moderate recall, which is expected because the simulated outdoor scene includes small airborne targets, bird distractors, clutter, and partial occlusions. In this work, YOLO is used as a candidate detector rather than the final decision module. The detections are later checked using confidence thresholding, radar agreement, optical-flow consistency, and EKF-based tracking, so false positives and uncertain detections can be reduced in the sensor-fusion stage. The detection performance is expected to improve with longer training, more epochs, and further dataset balancing.

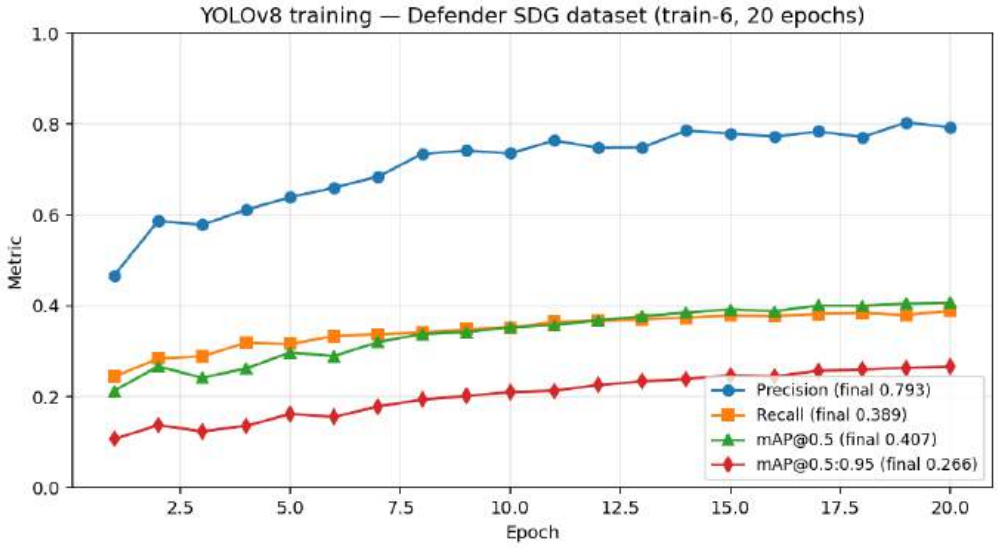


Figure 29: Training metrics versus epoch for YOLOv8n on the Defender SDG dataset.

Metric	Value
Precision	0.793
Recall	0.389
mAP@0.5	0.407
mAP@0.5:0.95	0.266

Table 15: YOLOv8n validation metrics after 20 epochs.

7.1.1 Optical-Flow Motion Consistency

Lucas–Kanade optical flow is computed on a region of interest in the Camera3 viewport. Flow estimates horizontal and vertical motion (`flow_u_px_s`, `flow_v_px_s`) and a residual (`flow_residual_px_s`) used to judge whether a YOLO box moves coherently with the scene.

For our log, optical flow was valid on 98.0% of frames. On every frame where YOLO was accepted (103 frames), `yolo_flow_consistent=1`. Thus, in this run, no accepted detection was rejected solely for flow disagreement.

Table 16: Optical-flow statistics in the closed loop (same engagement log).

Statistic	**Value**
Frames with `flow_valid=1`	98.0% of run
Mean residual when valid	16.3 px/s
95th percentile residual	29.2 px/s
Accepted YOLO with `yolo_flow_consistent=1`	100% of accepted frames
Configured residual cap	300 px/s
Configured angular tolerance	15°

The role of flow in the overall pipeline is therefore *guard against false visual associations* (e.g. birds or clutter boxes), while radar and the EKF carry continuous state estimation. Flow-derived Cartesian columns (`flow_meas_*_m`) are logged for offline analysis but are not the default live EKF input.

7.1.2 EKF-Based Target State Estimation

Figure 30 shows when each perception stage is active during the run. Radar remains in TRACK/LOCKED after handoff at $t \approx 0.2$ s; optical flow stays valid continuously; YOLO acceptance is intermittent (short bursts near $t \approx 2.5$–3.5 s, 5.5 s, and 8–9 s). EKF replay below uses radar–vision Cartesian measurements (`meas_*_m`), which are available whenever the track bus logs valid range ($z \geq 50$ m), independent of sporadic YOLO flags.

At frame k, the filter ingests `meas_x/y/z_m`; performance is the RMS error between the lead prediction at k and the sensor-frame reference at $k+L$ ($L = 2$ frames, ≈ 33 ms). Reference positions are reconstructed from logged line-of-sight data (`sensor_*_mrad`, `sensor_z_m`). The first 90 samples are skipped (≈ 1.5 s) to exclude handoff transients.

Table 17 summarizes radar-only replay (726 evaluation samples). Figure 31 compares predicted and true Cartesian trajectories; Figure 32 shows the equivalent angular and range errors.

Metric	x **/ azimuth**	y **/ elevation**	z **/ range**
Cartesian RMS @ $t+L$ [m]	3.70	6.04	4.26
Angular RMS @ $t+L$ [mrad]	0.95	2.80	—
Range RMS @ $t+L$ [m]	—	—	4.26
Hold meas. @ k vs truth @ $t+L$ [m]	9.98	14.46	10.43

Table 17: EKF lead prediction RMS — log, radar measurements only, $L = 2$

Relative to holding the current measurement, the EKF reduces RMS by approximately 63% (x), 58% (y), and 59% (z). Angular errors are smallest on the crossing (x) axis

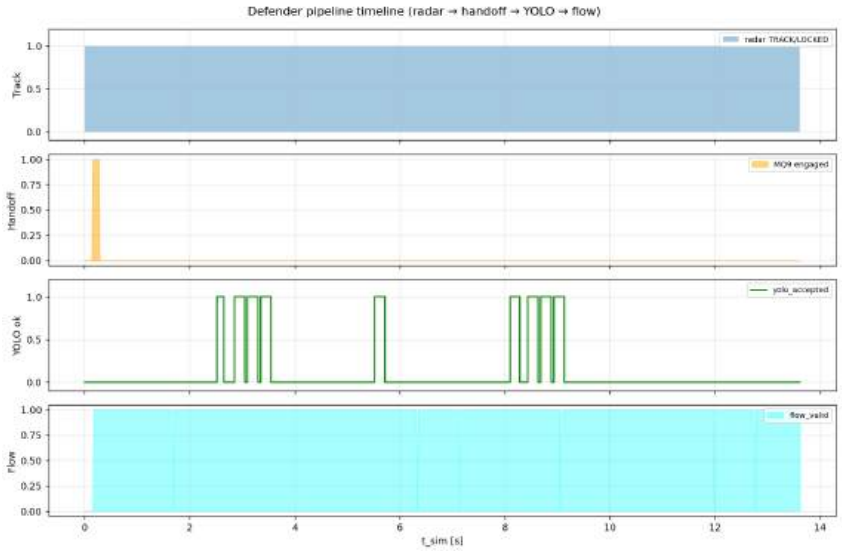


Figure 30: Defender pipeline timeline for log: radar track, MQ-9 handoff, YOLO acceptance, and optical-flow validity.

(0.95 mrad RMS); elevation-angle error reaches 2.80 mrad RMS with periodic spikes at trajectory reversals, mirrored in range error spikes of up to ±20 m while the range RMS remains 4.26 m.

Optical-Flow Fusion Note:

The same log includes `flow_meas_*_m` columns. Replaying with flow-valid fused measurements ($n = 710$) gives Cartesian RMS 3.75 m, 6.12 m, 4.04 m — a marginal change versus radar-only (Table 17) as shown in Figure 33. Optical flow is therefore treated as a YOLO validation gate in the live loop rather than the primary EKF input; reported results use radar–vision only.35).

7.2 Turret Tracking Performance

Figure 34 shows commanded vs. target azimuth and elevation; Figure 35 shows controller tracking error.

Over the full run, RMS tracking error is 4.36° (azimuth) and 12.58° (elevation), dominated by the startup transient. For $t \geq 4$ s (steady engagement), RMS improves to 3.59° (azimuth) and 1.79° (elevation). At the final frame ($t \approx 13.6$ s), errors are 3.5° azimuth and 3.3° elevation.

8 Conclusion

The automated Anti-Aircraft Defense System (AADS) project successfully achieves the rigorous performance benchmarks demanded by modern short-range air defense requirements. Through the systematic execution of advanced mechatronic principles, the system reliably bridges raw multi-modal sensor perception with high-torque mechanical actuation.

Key technical contributions and insights established during this research include:

- **Mechanical Alignment:** The developed mechanical subsystem aligns accurately

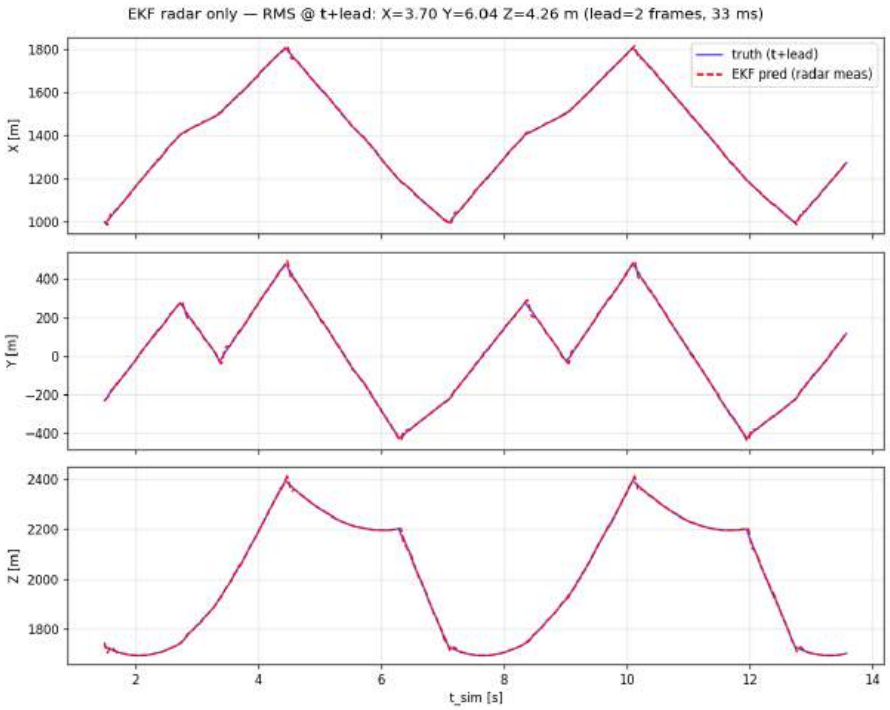


Figure 31: Radar-driven EKF: true vs. predicted Cartesian position at $t + L$ ($L = 2$, 33 ms). RMS: 3.70 m, 6.04 m, 4.26 m (x, y, z).

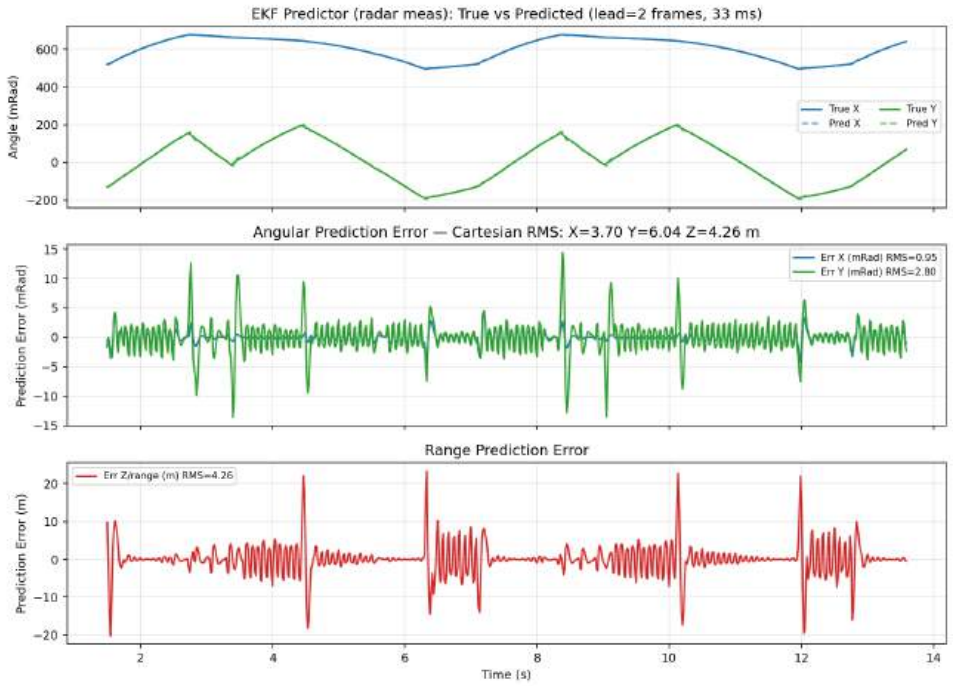


Figure 32: EKF predictor in sensor angular units (mrad) and range (m): true vs. predicted (top), angular errors (middle), range error (bottom). Cartesian RMS band matches Fig. 31.

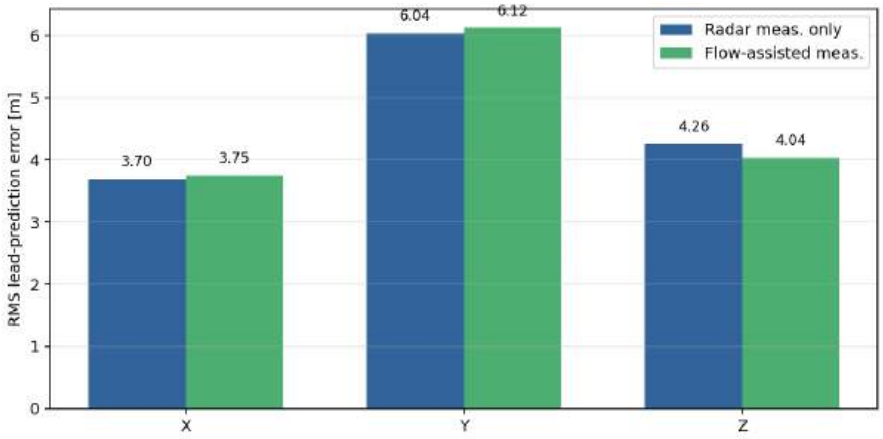


Figure 33: Comparison of EKF lead-prediction RMS error for radar-only measurements and optical-flow-assisted measurements.

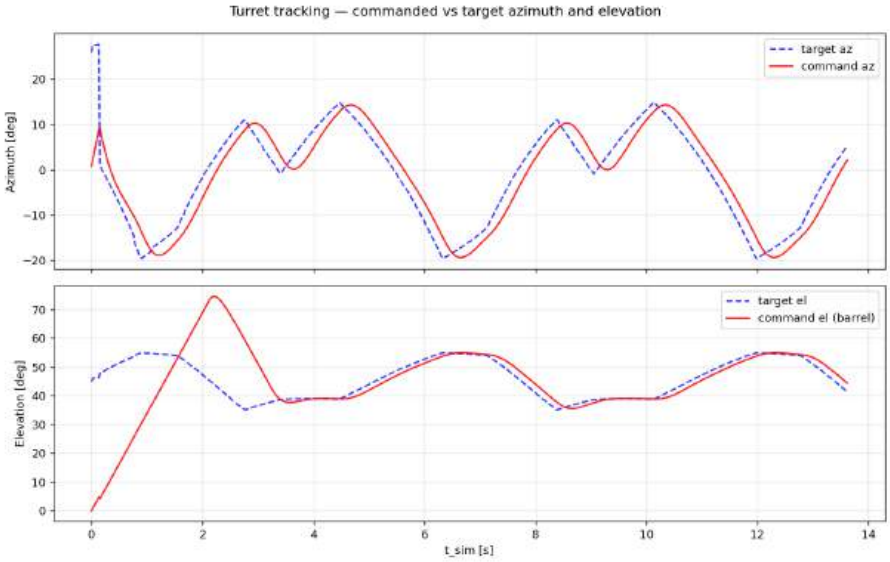


Figure 34: Turret command vs. target azimuth and elevation. Elevation exhibits a large initial transient ($t < 4\,\mathrm{s}$) before close tracking.

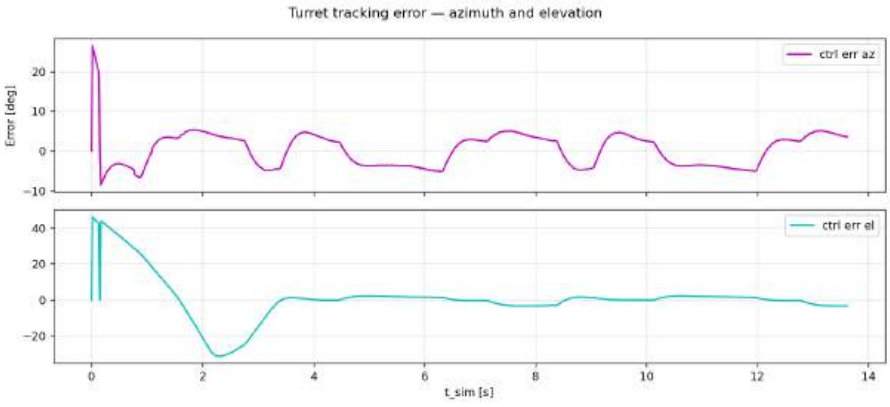


Figure 35: Controller tracking error (degrees). Azimuth settles after $\approx 1\,\mathrm{s}$; elevation after $\approx 4\,\mathrm{s}$.

with the physical and inertial characteristics of the baseline Leonardo OTO 76/62 gun mount architecture.

- **Control Strategy Performance:** The control strategy demonstrated effective performance under dynamic conditions, maintaining an average target miss distance of 1 meter.
- **Predictive Estimation Accuracy:** The EKF tracker successfully monitors and forecasts target trajectories, maintaining an angular prediction error margin well within < 10 mRad.
- **Machine Learning Target Recognition:** The applied deep learning model for automated aircraft recognition achieved an identification accuracy of 79%.
- **Rigorous Validation Testing:** Conducted exhaustive end-to-end validation testing to evaluate system performance and perception pipeline robustness across multiple environments.
- **Model Cross-Validation:** Completed a structural comparison between the continuous analytical mathematical model and the Simscape Multibody digital twin representation.
- **Worst-Case Actuator Assessment:** Evaluated critical motor characteristics under an aggressive, maximum-capability threat trajectory to guarantee operational feasibility.
- **Virtual War Scenario Testing:** Verified closed-loop target acquisition, sensor cueing, and gun tracking performance within a high-fidelity synthetic war scenario.

8.1 Future Work and Extensions

To advance the current baseline toward deployment-ready status, several high-value research directions are proposed:

1. **Multi-Modal Machine Learning Fusion:** Enhance target classification accuracy beyond the current 79.3% threshold by training a deep convolutional network that simultaneously convolves visible RGB frames, long-wave infrared (LWIR) thermal intensity maps, and raw Radar Cross Section (RCS) signatures.
2. **Dynamic Vessel Platform Simulation:** Transition the weapon system installation from a static ground mount to a dynamic orientation, expanding the Euler-Lagrange and EKF models to implement active 6-DOF ship motion compensation (roll, pitch, yaw, heave, sway, surge) for naval operations.
3. **Internal Ballistics and Predictive Interception:** Integrate explicit internal ballistic physics within the simulation loop, modeling projectile muzzle velocity, barrel heating, rifling wear, and explosive payloads to establish an exact automated fire-control trigger.
4. **Intelligent Guidance Closures:** Explore the integration of adaptive closed-loop projectile dynamics—such as canard-guided smart ammunition or real-time course correction telemetry—to expand the effective terminal engagement envelope against highly evasive supersonic threats.